



UNIVERSIDAD TÉCNICA DEL NORTE
FACULTAD DE INGENIERÍA EN CIENCIAS APLICADAS
CARRERA DE TECNOLOGÍAS DE LA INFORMACIÓN

TRABAJO DE INTEGRACIÓN CURRICULAR PREVIO A LA OBTENCIÓN DEL
TÍTULO DE INGENIERO EN TECNOLOGÍAS DE LA INFORMACIÓN

**” GENERACIÓN AUTOMÁTICA DE REQUISITOS FUNCIONALES
DE SOFTWARE MEDIANTE INTELIGENCIA ARTIFICIAL
GENERATIVA Y ARQUITECTURA MULTI-AGENTE. ”**



AUTOR: Haro Osnayo Andrés Sebastian
DIRECTOR: Dr. Guevara Vega Cathy Pamela

Ibarra – Ecuador
2026

UNIVERSIDAD TÉCNICA DEL NORTE
BIBLIOTECA UNIVERSITARIA

IDENTIFICACIÓN DE LA OBRA

La Universidad Técnica del Norte dentro del proyecto Repositorio Digital Institucional, determinó la necesidad de disponer de textos completos en formato digital con la finalidad de apoyar los procesos de investigación, docencia y extensión de la Universidad.

Por medio del presente documento dejo sentada mi voluntad de participar en este proyecto, para lo cual pongo a disposición la siguiente información:

DATOS DE CONTACTO			
CÉDULA DE IDENTIDAD:	1005158637		
APELLIDOS Y NOMBRES:	HARO OSNAYO ANDRÉS SEBASTIAN		
DIRECCIÓN:	COTACACHI		
EMAIL:	ASHAROO@UTN.EDU.EC		
TELF. FIJO:	99999	TELF. MÓVIL:	0939951645

DATOS DE LA OBRA	
TÍTULO:	GENERACIÓN AUTOMÁTICA DE REQUISITOS FUNCIONALES DE SOFTWARE MEDIANTE INTELIGENCIA ARTIFICIAL GENERATIVA Y ARQUITECTURA MULTI-AGENTE.
AUTOR(ES):	HARO OSNAYO ANDRÉS SEBASTIAN
FECHA:	Escriba la fecha en la que terminó su trabajo de titulación en formato dd/MM/yyyy
CARRERA/PROGRAMA:	PREGRADO
TÍTULO POR EL QUE OPTA:	INGENIERO EN TECNOLOGÍAS DE LA INFORMACIÓN
DIRECTOR:	DR. GUEVARA VEGA CATHY PAMELA

AUTORIZACIÓN DE USO A FAVOR DE LA UNIVERSIDAD

Yo, Haro Osnayo Andrés Sebastian , con cédula de identidad Nro. 1005158637 , en calidad de autor (es) y titular (es) de los derechos patrimoniales de la obra o trabajo de integración curricular descrito anteriormente, hago entrega del ejemplar respectivo en formato digital y autorizo a la Universidad Técnica del Norte, la publicación de la obra en el Repositorio Digital Institucional y uso del archivo digital en la Biblioteca de la Universidad con fines académicos, para ampliar la disponibilidad del material y como apoyo a la educación, investigación y extensión; en concordancia con la Ley de Educación Superior Artículo 144.

Escriba la fecha en la que terminó su trabajo de titulación en formato dd/MM/yyyy

EL AUTOR:

Firma Aquí

Firma

Nombre Haro Osnayo Andrés Sebastian

CONSTANCIAS

El autor (es) manifiesta (n) que la obra objeto de la presente autorización es original y se la desarrolló, sin violar derechos de autor de terceros, por lo tanto, la obra es original y que es (son) el (los) titular (es) de los derechos patrimoniales, por lo que asume (n) la responsabilidad sobre el contenido de la misma y saldrá (n) en defensa de la Universidad en caso de reclamación por parte de terceros.

Escriba la fecha en la que terminó su trabajo de titulación en formato dd/MM/yyyy

EL AUTOR:

Firma Aquí

Firma

Nombre Haro Osnayo Andrés Sebastian

CERTIFICACIÓN DEL DIRECTOR DEL TRABAJO DE INTEGRACIÓN CURRICULAR

Escriba la fecha en la que terminó su trabajo de titulación en formato dd/MM/yyyy

Dr. Guevara Vega Cathy Pamela
DIRECTOR DEL TRABAJO DE INTEGRACIÓN CURRICULAR

CERTIFICA:

Haber revisado el presente informe final del trabajo de Integración Curricular, el mismo que se ajusta a las normas vigentes de la Universidad Técnica del Norte; en consecuencia, autorizo su presentación para los fines legales pertinentes.

Director

Dr. Guevara Vega Cathy Pamela

C.C: *Cédula de Identidad*

APROBACIÓN DEL COMITÉ CALIFICADOR

El Comité Calificado del trabajo de Integración Curricular “ Generación automática de requisitos funcionales de software mediante inteligencia artificial generativa y arquitectura Multi-agente. ” elaborado por Haro Osnayo Andrés Sebastian , previo a la obtención del título del INGENIERO EN TECNOLOGÍAS DE LA INFORMACIÓN , aprueba el presente informe de investigación en nombre de la Universidad Técnica del Norte:

Director

Dr. Guevara Vega Cathy Pamela

C.C: *Cédula de Identidad*

Asesor

MSc. Pablo Andrés Landeta

C.C: *Cédula de Identidad*

DEDICATORIA

Haro Osnayo Andrés Sebastian

AGRADECIMIENTO

Haro Osnayo Andrés Sebastian

ÍNDICE DE CONTENIDOS

DEDICATORIA	7
AGRADECIMIENTO	8
ÍNDICE DE FIGURAS	14
ÍNDICE DE TABLAS	15
RESUMEN	16
ABSTRACT	17
CAPÍTULO I	
1.1 Problema	18
1.2 Objetivos	20
1.2.1 Objetivo general	20
1.2.2 Objetivos específicos.	20
1.3 Alcance	20
1.4 Justification	23
CAPÍTULO II	
2.1 Antecedentes	26
2.2 Ingeniería de Requisitos	31
2.2.1 Metodología de Elicitación de Requisitos.	32
2.2.2 Estándar IEEE 830-1998.	33
2.2.3 Tipos de requisitos: funcionales	33
2.3 Inteligencia Artificial aplicada a la Ingeniería de Software	34

2.3.1	Modelos de Lenguaje de Gran Escala (LLMs)	35
2.3.2	Limitaciones de los LLMs.	37
2.3.3	Retrieval-Augmented Generation (RAG)	38
2.4	Sistemas Multi-agente	39
2.4.1	Definición y características de los agentes de IA	39
2.4.2	Arquitecturas multi-agente basadas en LLMs.	40
2.4.3	Coordinación y comunicación entre agentes.	41
2.4.4	Frameworks para sistemas multi-agente	42
2.5	Model Context Protocol (MCP)	43
2.5.1	Definición y propósito del MCP	43
2.5.2	Arquitectura cliente-servidor.	44
2.5.3	Resources, Tools y Prompts	45
2.5.4	Integración del MCP con agentes de IA	47
2.6	Reconocimiento Automático de Voz (ASR)	48
2.6.1	Fundamentos del ASR.	48
2.6.2	Modelos open source para transcripción.	49
2.6.3	ASR aplicado a entrevistas de elicitación	50
2.6.4	Integración del ASR en sistemas multi-agente	51
2.7	Arquitectura e Integración de Aplicaciones Web	52

2.7.1	Arquitectura de aplicaciones web	52
2.7.2	Integración entre frontend, backend y agentes	53
2.7.3	Visualización de resultados y reportes	54
CAPÍTULO III		
3.1	Descripción general	56
3.2	Arquitectura del sistema	56
3.2.1	Arquitectura general del sistema multiagente	57
3.2.2	Componentes y responsabilidades	58
3.2.3	Flujo de procesamiento de la información	59
3.2.4	Diseño de la base de datos.	60
3.3	Implementación del protocolo MCP	62
3.3.1	Organización de la integración MCP	62
3.3.2	Cliente MCP y comunicación con los servidores	62
3.3.3	Servidor MCP de consulta académica.	64
3.3.4	Servidor MCP de base de datos.	64
3.3.5	Servidor MCP de generación SRS	64
3.4	Implementación de los agentes especializados y del orquestador	66
3.4.1	Agente de transcripción.	66
3.4.2	Agente de extracción y clasificación de requisitos funcionales	67
3.4.3	Agente de enriquecimiento académico por proyecto	68
3.4.4	Agente generador del documento SRS	68
3.4.5	Orquestador del sistema	69
3.5	Tecnologías y entorno de ejecución	71

3.5.1	Características del equipo utilizado.	71
3.5.2	Entorno de software y herramientas utilizadas	72
3.5.3	Configuración de ejecución.	73
3.6	Metodología de desarrollo: adaptación de Scrum al proyecto	74
3.6.1	Adaptación de Scrum al desarrollo individual	74
3.6.2	Responsabilidades de los participantes	75
3.6.3	Definición del Product Backlog	75
3.6.4	Seguimiento y revisión de los incrementos.	78
3.7	Planificación y desarrollo de los sprints	78
3.7.1	Sprint 1: configuración y transcripción.	80
3.7.2	Sprint 2: extracción de requisitos y orquestador inicial	81
3.7.3	Sprint 3: integración MCP y enriquecimiento académico	81
3.7.4	Sprint 4: generación del documento SRS	82
3.7.5	Sprint 5: interfaz, integración y pruebas	83
3.8	Estructura del proyecto	84
3.8.1	Organización de directorios y archivos.	85
3.8.2	Separación de responsabilidades en el código	87
3.9	Dificultades técnicas y soluciones implementadas	88
3.9.1	Control de la extracción de requisitos.	88
3.9.2	Ajustes en el enriquecimiento académico.	88
3.9.3	Delimitación del contenido del documento SRS.	89
CAPÍTULO IV		
CONCLUSIONES		91

RECOMENDACIONES	92
BIBLIOGRAFÍA	93
ANEXOS	98

ÍNDICE DE FIGURAS

Figura 1	Análisis causal de la generación manual de documentos SRS	19
Figura 2	Arquitectura del sistema multi-agente	22
Figura 3	Arquitectura lógica de la simbiosis futura entre ingenieros de software e IA[16].	35
Figura 4	Proceso general de funcionamiento de un LLM [28].	36
Figura 5	Ejemplo de arquitectura multi-agente basada en roles para generación y priorización de historias de usuario [15].	40
Figura 6	Flujo de trabajo del Model Context Protocol [23].	46
Figura 7	Flujo de procesamiento para la generación del documento SRS.	60
Figura 8	Diseño de la base de datos	61
Figura 9	Secuencia de comunicación para la ejecución de herramientas MCP. . .	63
Figura 10	Diagrama de actividades del procesamiento multiagente.	70
Figura 11	Organización del desarrollo del prototipo mediante cinco sprints	79
Figura 12	Estructura de directorios del prototipo.	85
Figura 13	Estructura de servicios	86

ÍNDICE DE TABLAS

Tabla 1	Componentes principales del prototipo	58
Tabla 2	Principales herramientas MCP	65
Tabla 3	Herramientas y modelos utilizados por los agentes y el orquestador	66
Tabla 4	Características del equipo utilizado	71
Tabla 5	Software y modelos utilizados en el prototipo	72
Tabla 6	Participantes y responsabilidades en el proyecto	75
Tabla 7	Product Backlog del prototipo	76
Tabla 8	Distribución de los sprints	79
Tabla 9	Tareas y criterios de aceptación del sprint 1	80
Tabla 10	Tareas y criterios de aceptación del sprint 2	81
Tabla 11	Tareas y criterios de aceptación del sprint 3	82
Tabla 12	Tareas y criterios de aceptación del sprint 4	83
Tabla 13	Tareas y criterios de aceptación del sprint 5	84
Tabla 14	Responsabilidades de los directorios y archivos principales	87

RESUMEN

El resumen debe tener 250 palabras como mínimo, su resumen tiene 0 caracteres está fuera del rango de 250 palabras

Palabras clave: Palabra 1 , Palabra 2 , Palabra 3

ABSTRACT

El resumen debe tener 250 palabras como mínimo, su resumen tiene 0 caracteres está fuera del rango de 250 palabras

Keywords: Word 1 , Word 2, Word 3

CAPÍTULO I

INTRODUCCIÓN

1.1 Problema

La Ingeniería de Requisitos (RE) constituye una fase crítica del ciclo de vida del desarrollo de software, cuyo objetivo es garantizar que las necesidades de todos los interesados sean capturadas, analizadas y documentadas con precisión antes de iniciar la construcción del sistema [1]. Asegurar la calidad de estas especificaciones representa un desafío importante debido a su carácter no estructurado y a la dependencia de expertos humanos para su elaboración [1]. En la práctica actual, la creación del documento de Especificación de Requisitos de Software (SRS) sigue siendo una tarea predominantemente manual que consume tiempo excesivo, recayendo sobre los miembros más experimentados del equipo e impidiendo que destinen su esfuerzo a actividades de mayor valor [2].

Esta situación se origina en la naturaleza ambigua del lenguaje natural empleado en las entrevistas, limitación reconocida explícitamente por el estándar IEEE 830-1998, que establece la necesidad de revisión independiente para identificar usos ambiguos antes de aceptar un SRS como válido [3], [4]. La ausencia de herramientas que guíen la estructuración formal del documento genera especificaciones incompletas, inconsistentes o no verificables [4], dificultando especialmente la extracción de requisitos funcionales a partir de conversaciones informales, cuya omisión compromete atributos de calidad como rendimiento, seguridad y Conservabilidad [5], [6]. A pesar del avance de los Modelos de Lenguaje de Gran Escala (LLMs), su uso individual presenta limitaciones en la gestión de contextos extensos y propensión a generar información incorrecta denominada "alucinaciones", agravando aún más el problema [2].

Las consecuencias de una documentación deficiente son severas: los déficits detectados en

fases tardías provocan retrabajos costosos que comprometen presupuesto y plazos [1], debilitan la comunicación entre clientes y desarrolladores, e incrementan el riesgo de fracaso del proyecto [5]. En este proceso intervienen variables críticas como la calidad semántica de los requisitos, completitud, consistencia y ausencia de ambigüedad[7], el tiempo invertido en la redacción del SRS y el grado de conformidad con el estándar IEEE 830-1998 [4], [8].

Desde el campo de la ingeniería, se propone el desarrollo de un sistema multi-agente basado en el Model Context Protocol e inteligencia artificial generativa, que especializa los LLMs en roles diferenciados de transcripción, análisis, clasificación y generación del documento SRS [9], superando las limitaciones de los modelos individuales mediante inyección de contexto rico proveniente del audio de entrevistas y comunicación estructurada entre agentes y fuentes de datos externas [10]. Este enfoque busca automatizar la transición entre el lenguaje natural hablado y la especificación técnica formal conforme al estándar IEEE 830-1998, reduciendo la carga operativa del equipo de ingeniería y elevando la calidad de la documentación producida.

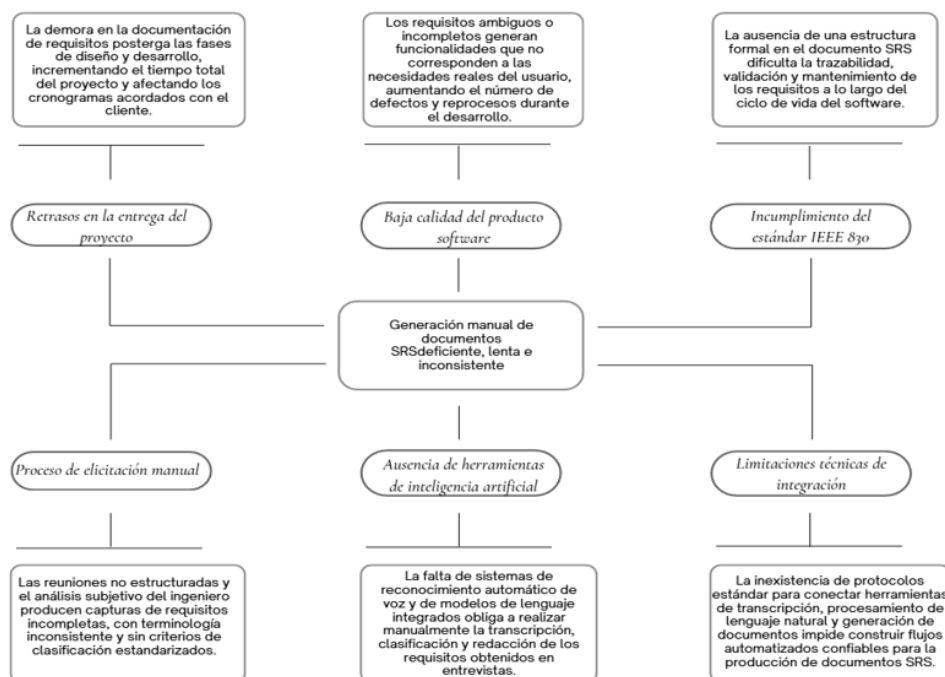


Figura 1.
Análisis causal de la generación manual de documentos SRS

1.2 Objetivos

1.2.1 Objetivo general

Desarrollar un sistema multi-agente basado en Model Context Protocol, para la generación automatizada de documentos de especificación de requisitos de software conforme al estándar IEEE 830-1998, a partir de audios de entrevistas a Product Owners.

1.2.2 Objetivos específicos

- Establecer el marco teórico y conceptual que fundamente la investigación, abordando los fundamentos de ingeniería de requisitos, metodologías de elicitación, el estándar IEEE 830-1998, modelos de lenguaje de gran escala, arquitecturas multi-agente y el MCP.
- Desarrollar el sistema multi-agente basado en MCP e inteligencia artificial generativa, mediante la metodología de desarrollo Scrum, integrando agentes especializados para la transcripción de audio de entrevistas, extracción y clasificación de requisitos funcionales, enriquecimiento contextual mediante búsqueda web, y generación estructurada del documento SRS conforme al estándar IEEE 830-1998.
- Evaluar los documentos SRS generados por el sistema mediante métricas basadas en LLMs y juicio de expertos en ingeniería de requisitos, aplicando como criterios de validación, comparando los resultados obtenidos frente a la elaboración manual del documento.

1.3 Alcance

El presente trabajo comprende el diseño y evaluación de un prototipo funcional de sistema multi-agente basado en MCP, orientado a la generación automatizada de documentos de SRS conformes al estándar IEEE 830-1998, a partir del procesamiento de audio de entrevistas a Product Owners.

En primer lugar, se elaborará el marco teórico mediante el estudio y análisis de literatura científica relacionada con ingeniería de requisitos, la metodología de elicitación, el estándar IEEE 830-1998, modelos de lenguaje de gran escala, arquitecturas multi-agente y el MCP, estableciendo los fundamentos conceptuales que sustentan el sistema propuesto.

Posteriormente, se realizará el diseño y desarrollo del sistema, el cual integrará cuatro agentes: un agente de transcripción automática del audio de la entrevista, un agente de análisis y clasificación de requisitos funcionales, un agente de enriquecimiento contextual mediante búsqueda web, y un agente generador del documento SRS estructurado conforme al estándar IEEE 830-1998. El desarrollo del sistema se gestionará mediante la metodología Scrum, organizando el trabajo en sprints con entregas incrementales que permitan una evolución controlada y adaptable del proyecto. El sistema incorporará mecanismos de persistencia en base de datos para almacenar los documentos generados y permitir su consulta y comparación futura, así como una interfaz web que permita cargar el audio de la entrevista y visualizar el documento SRS producido. Finalmente, la evaluación del sistema se realizará mediante métricas basadas en modelos de lenguaje de gran escala (LLMs) y juicio de expertos en ingeniería de requisitos, quienes validarán la conformidad, completitud y coherencia de los documentos SRS generados respecto al estándar IEEE 830-1998 generando resultados comparativos que permitan determinar la efectividad del sistema frente a la elaboración manual del documento.

Cabe señalar que el proyecto se limita al desarrollo de un prototipo funcional con fines académicos e investigativos, por lo que no contempla su integración con plataformas, ni su despliegue en entornos productivos. El entrenamiento o ajuste fino de modelos de lenguaje queda fuera del alcance, utilizándose exclusivamente modelos pre entrenados disponibles mediante API.

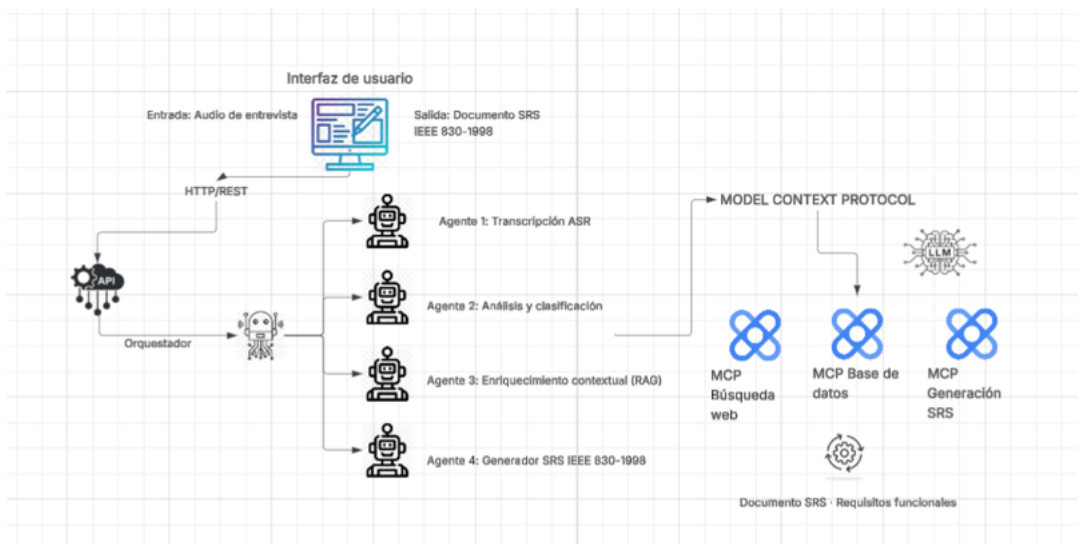


Figura 2.
Arquitectura del sistema multi-agente

El sistema inicia cuando el usuario carga el audio de la entrevista de elicitación a través de la interfaz web. Esta interfaz se comunica con el backend mediante protocolo HTTP/REST, enviando el audio al componente central del sistema.

La petición es recibida por la API, que actúa como punto de entrada y la redirige al Agente Orquestador. Este agente es el responsable de coordinar y dirigir el flujo de trabajo entre los cuatro agentes especializados, asegurando que cada uno ejecute su tarea en el orden correcto. Los cuatro agentes especializados operan de la siguiente manera:

El Agente 1 de Transcripción ASR recibe el audio de la entrevista y lo convierte en texto mediante reconocimiento automático de voz. El Agente 2 de Análisis y Clasificación procesa el texto transcrito para identificar y clasificar los requisitos funcionales del sistema. El Agente 3 de Enriquecimiento Contextual (RAG) amplía y enriquece el contexto de los requisitos identificados mediante búsqueda de información relevante en la web. Finalmente, el Agente 4 Generador SRS toma todos los requisitos procesados y enriquecidos para construir el documento SRS estructurado conforme al estándar IEEE 830-1998.

La comunicación entre los agentes y los servicios externos se gestiona a través del MCP, el cual conecta el sistema con tres servidores especializados: el MCP de Búsqueda Web, que provee

información contextual al Agente 3; el MCP de Base de Datos, que almacena los documentos SRS generados para su consulta y comparación futura; y el MCP de Generación SRS, que apoya la construcción estructurada del documento final. Adicionalmente, todos los agentes hacen uso de un modelo de lenguaje de gran escala (LLM) para el procesamiento y generación de texto.

Como resultado final, el sistema devuelve el documento SRS completo, el cual incluye los requisitos funcionales organizados conforme al estándar IEEE 830-1998.

1.4 Justification

El presente trabajo constituye una contribución original al campo de la ingeniería de software al integrar tres áreas de investigación de frontera: los modelos de lenguaje de gran escala, las arquitecturas multi-agente y el MCP, aplicados a la automatización de un proceso tan fundamental como la especificación de requisitos de software. Académicamente, el sistema propuesto aporta un marco de referencia implementable para futuras investigaciones que busquen extender la automatización hacia otras fases del ciclo de vida del desarrollo de software. Asimismo, la metodología de evaluación basada en juicio de expertos con criterios derivados del estándar IEEE 830-1998 y la metodología de elicitación establece un precedente metodológico reproducible para trabajos similares en el ámbito hispanohablante, donde la documentación técnica en español representa un desafío adicional para los sistemas de procesamiento de lenguaje natural.

Desde la perspectiva investigativa, este trabajo responde directamente a las brechas identificadas en la literatura científica reciente. Wolf et al. [1] señalan que el campo de la ingeniería de requisitos asistida por IA carece de integración con procesos reales de ingeniería de software y de estandarización metodológica. El sistema propuesto aborda precisamente esta brecha al conectar el proceso real de elicitación —la entrevista de audio— con la generación formal del documento SRS bajo un estándar reconocido internacionalmente. En este contexto, investigaciones previas desarrolladas en la Universidad Técnica del Norte han demostrado la viabilidad de aplicar LLMs en procesos de análisis automatizado de documentos técnicos [11], [12] validando la pertinencia de esta línea de investigación en el contexto institucional y estableciendo

un antecedente directo sobre el que la presente propuesta busca avanzar. Adicionalmente, la incorporación del MCP como infraestructura de comunicación entre agentes representa una línea de investigación emergente con escasa exploración en el contexto de la ingeniería de requisitos, lo que posiciona este trabajo en la frontera del conocimiento actual en sistemas multiagente basados en LLMs[9], [10]. Técnicamente, el sistema aportará una solución concreta y funcional para uno de los problemas más costosos del desarrollo de software: la elaboración manual del documento SRS. La arquitectura multi-agente propuesta permite distribuir la complejidad del proceso en agentes especializados, superando las limitaciones de los modelos de lenguaje individuales en términos de gestión de contextos extensos y propensión a generar alucinaciones. [2]. La integración del agente de búsqueda web mediante RAG (Retrieval-Augmented Generation) enriquecerá el contexto del dominio específico del software a desarrollar, mejorando la pertinencia y completitud de los requisitos generados [6]. El sistema también aportará un repositorio persistente de documentos SRS que permitirá la consulta, comparación y reutilización de requisitos entre proyectos, constituyendo una base de conocimiento organizacional de alto valor para equipos de desarrollo. Desde el punto de vista económico, la automatización de la generación del documento SRS representa un ahorro significativo en horas de trabajo de los ingenieros más experimentados del equipo, quienes actualmente destinan una parte considerable de su tiempo a tareas de transcripción, clasificación y redacción formal de requisitos [2].

Este tiempo liberado puede ser redirigido hacia actividades de mayor valor como el diseño de arquitectura, la revisión de código o la gestión de riesgos del proyecto. Para organizaciones de desarrollo de software de tamaño pequeño y mediano, donde los recursos humanos son limitados, esta reducción en el esfuerzo de documentación puede representar una ventaja competitiva significativa y una reducción directa en los costos asociados a retrabajos provocados por especificaciones deficientes [1], [5].

Socialmente, el sistema beneficiará a múltiples actores del ecosistema de desarrollo de software. Los ingenieros de software verán reducida su carga operativa en tareas repetitivas y podrán enfocarse en actividades que requieren mayor creatividad y juicio profesional. Los usuarios finales y clientes se beneficiarán de documentos de requisitos más completos y consistentes, lo que

reduce la probabilidad de que el sistema construido no satisfaga sus necesidades reales [4]. En el ámbito educativo, el sistema tiene un potencial pedagógico significativo: estudiantes de ingeniería de software podrán utilizarlo como herramienta de apoyo para comprender la estructura y contenido de un documento SRS conforme al estándar IEEE 830-1998, aprendiendo de manera práctica los elementos que debe contener una especificación de calidad [3], [4]. La metodología de elicitación de Durán y Bernárdez [13], ampliamente utilizada en la enseñanza de ingeniería de requisitos en el mundo hispanohablante, servirá como referente pedagógico integrado en el sistema.

En síntesis, el sistema propuesto resolverá un problema concreto y medible: la brecha existente entre el lenguaje natural de las entrevistas de elicitación y la especificación técnica formal requerida por el estándar IEEE 830-1998. Sus beneficiarios directos son los ingenieros de software, equipos de desarrollo, organizaciones de tecnología y estudiantes de ingeniería. Sus aportes técnicos incluyen una arquitectura multi-agente replicable, un flujo automatizado de procesamiento desde el audio de la entrevista hasta la generación del documento SRS, y un conjunto de prompts estructurados para la extracción y clasificación de requisitos. Sus aportes teóricos incluyen evidencia empírica sobre la viabilidad del uso de LLMs multi-agente con MCP en la automatización de procesos de ingeniería de requisitos, contribuyendo al avance del conocimiento en un área de alta relevancia para la industria del software a nivel global.

CAPÍTULO II

MARCO TEÓRICO

2.1 Antecedentes

Puchleitner et al. [14] abordaron la problemática de la ineficiencia y la alta demanda de tiempo que caracteriza a la Ingeniería de Requisitos (IR), proceso crítico cuyo manejo deficiente suele comprometer sistemáticamente el éxito de los proyectos de software. El estudio tuvo como objetivo evaluar el desempeño de los modelos de lenguaje de gran tamaño (LLM) en la generación de especificaciones funcionales coherentes y contextualmente relevantes a partir del alcance de un proyecto. Para ello, se aplicó una metodología comparativa analítica utilizando los modelos GPT-4o y Llama-3-70b-instruct sobre tres proyectos del dataset PURE, empleando una estructura de prompt refinada que definía roles especializados y priorizaba las salidas por nivel de importancia. Los resultados revelaron que aproximadamente el 80 % de los requisitos generados por ambos modelos poseían relevancia funcional, logrando un solapamiento de contenido superior al 88 % en aquellos clasificados con mayor prioridad. Asimismo, el análisis crítico detectó que las divergencias semánticas y la inclusión de requisitos implícitos representan los principales desafíos para alcanzar una alineación total con las especificaciones humanas. Este trabajo sustenta la presente investigación al validar empíricamente que la IA generativa puede automatizar tareas de documentación técnica inicial con alta precisión, sentando un precedente fundamental para el desarrollo de sistemas que busquen optimizar la fase de especificación mediante el uso de arquitecturas inteligentes [14].

Sami et al. [15] abordaron la ineficiencia y falta de consistencia inherente a la especificación y priorización manual de historias de usuario en marcos de trabajo ágiles. El estudio tuvo como propósito evaluar la eficacia de un sistema multi-agente basado en roles, implementando

modelos de lenguaje de gran tamaño (LLM) como GPT-4o y Llama 3.3 para automatizar estos procesos críticos. La metodología consistió en simular la interacción de un equipo de desarrollo conformado por agentes especialistas (Dueño del Producto, Desarrollador y QA), quienes emplearon la técnica de los 100 dólares para clasificar los requisitos generados a partir de la visión de un proyecto logístico real [15]. Los resultados evidenciaron que, si bien todos los modelos producen requisitos con alta relevancia funcional y similitud semántica, GPT-4o demostró una cobertura más exhaustiva de las fases del proyecto en comparación con sus pares [15]. Asimismo, el análisis crítico reveló que la alineación con el juicio experto humano es moderada, enfrentando dificultades para capturar contextos de dominio dinámicos y matizados [15]. Por otra parte, se concluyó que la variabilidad entre ejecuciones persiste como un desafío técnico para la estabilidad de las propuestas generadas. Esta investigación sustenta la presente tesis al validar el potencial de las arquitecturas multi-agente para orquestar perspectivas técnicas diversas en la Ingeniería de Requisitos, subrayando a la vez la necesidad de refinar los mecanismos de integración de contexto para alcanzar una autonomía superior en la documentación SRS [15].

Terragni et al. [16] analizan la transición hacia una ingeniería de software potenciada por IA, enfrentando el desafío de gestionar la creciente complejidad de los sistemas mediante niveles superiores de abstracción. El estudio tiene como objetivo proponer un marco conceptual para una colaboración simbiótica entre desarrolladores y sistemas multi-agente que optimice fases críticas como la ingeniería de requisitos. Mediante una metodología prospectiva basada en el análisis de herramientas *“agentic”* actuales, los autores describen una arquitectura donde agentes especializados, coordinados por un orquestador, ejecutan tareas de elicitación, validación y resumen de especificaciones funcionales. Los resultados sugieren que la evolución tecnológica conduce inevitablemente hacia sistemas autónomos capaces de percibir y actuar en el entorno de desarrollo, aunque se concluye que la supervisión humana es vital para garantizar la confiabilidad frente a las alucinaciones del modelo [16]. Asimismo, este trabajo fundamenta la presente tesis al proporcionar una base arquitectónica robusta que justifica el uso de agentes orquestados para la generación y verificación automática de documentación SRS profesional [16].

Nguyen-Duc et al. [17] abordan la carencia de una visión integral sobre los desafíos y apli-

caciones prácticas de la Inteligencia Artificial Generativa (GenAI) en la Ingeniería de Software. El estudio se propuso definir una agenda de investigación mediante una metodología que integró una revisión de literatura con cuatro grupos focales de expertos durante un periodo de cinco meses. Entre los resultados principales, se identificaron 78 preguntas de investigación abiertas, subrayando que áreas críticas como la ingeniería de requisitos han recibido menor atención frente a la implementación de código, enfrentando obstáculos como alucinaciones, sesgos y limitaciones en el procesamiento de contexto global [17]. Asimismo, los autores resaltan la visión de utilizar agentes de IA basados en roles para la automatización de proyectos, lo cual guarda una relación directa con esta investigación al fundamentar la transición hacia arquitecturas multi-agente. Por otra parte, el estudio concluye que el futuro de la disciplina radica en plataformas de colaboración simbiótica humano-IA, sustentando la pertinencia de automatizar la generación de requisitos para optimizar la productividad y la calidad documental [17].

Mahbub et al. [18] abordaron la problemática de los defectos en especificaciones de requisitos de software (SRS) industriales, cuya detección manual resulta ineficiente en documentos extensos y complejos. El objetivo del estudio fue evaluar el desempeño de GPT-4, bajo un enfoque de aprendizaje de disparo cero (zero-shot), en la identificación de ambigüedades, inconsistencias e incompletitud dentro de un SRS de 52 páginas para un sistema de ventilación mecánica. La metodología empleada incluyó el procesamiento de segmentos textuales y descripciones de diagramas UML, permitiendo comparar los hallazgos de la IA con el criterio de expertos humanos. Los resultados destacaron una precisión de 0.61 para detectar omisiones, aunque el rendimiento decreció significativamente en inconsistencias (0.43) y ambigüedades (0.39) [18]. El análisis concluyó que, si bien la IA facilita una detección temprana y recupera el 60 % de los fallos identificados por especialistas, presenta limitaciones críticas en la gestión del contexto global y el conocimiento de dominio profundo [18]. Asimismo, esta investigación es fundamental para la presente tesis, ya que evidencia la necesidad de implementar mecanismos de validación automática que superen las restricciones de contexto detectadas mediante arquitecturas más sofisticadas.

Binkhonain y Alfayaz [19] abordan la problemática de la dependencia de grandes conjun-

tos de datos etiquetados y la escasa capacidad de generalización de los modelos supervisados tradicionales en la clasificación de requisitos de software. El objetivo del estudio fue evaluar la efectividad de los modelos de lenguaje de gran tamaño (LLM) y diversas técnicas de prompting para automatizar esta tarea crítica. La metodología consistió en un estudio de referencia empírico que comparó cinco modelos, tales como GPT-4 Turbo y Gemini 2.0 Flash, aplicando estrategias de zero-shot, few-shot, persona y cadena de pensamiento sobre los datasets PROMISE y SecReq. Los resultados evidenciaron que la configuración few-shot, potenciada con la asignación de roles, logra un rendimiento superior o comparable al de modelos especializados como BERT [19]. Asimismo, se concluyó que el uso de LLMs basados en instrucciones ofrece una solución flexible y escalable que minimiza la necesidad de reentrenamiento manual. Por otra parte, este trabajo sustenta la presente tesis al validar técnicamente el uso de IA generativa para el tratamiento de requisitos, demostrando que la especialización instruccional es un pilar fundamental para la automatización de la documentación funcional y técnica [19].

Maestre Peña et al. [20] abordan la ineficiencia en el procesamiento del masivo volumen de retroalimentación de usuarios en plataformas digitales para la Ingeniería de Requisitos. El objetivo del estudio fue desarrollar un método híbrido que integra modelos de Aprendizaje Profundo para el filtrado de datos y Modelos de Lenguaje de Gran Tamaño (LLM) para la especificación automática de requisitos. La metodología implementada consistió en tres fases: clasificación de opiniones informativas mediante BERTweet con inyección de conocimiento técnico (ISO/IEC/IEEE 24765), agrupamiento temático basado en Fuzzy C-Means y generación mediante DeepSeek V3 utilizando un prompt estructurado por etapas. Los resultados obtenidos en cuatro conjuntos de datos reales evidenciaron un rendimiento superior a soluciones estatales, logrando un F1-Score promedio de 0.950 y garantizando la trazabilidad hacia la fuente original [20]. Asimismo, el estudio concluye que la calidad de la documentación generada depende críticamente de la organización y el filtrado previo de la información contextual. Por otra parte, este antecedente es fundamental para la presente investigación, ya que valida la efectividad de los LLM en la transformación de lenguaje natural informal en especificaciones funcionales bajo criterios de calidad, sentando las bases técnicas para el uso de arquitecturas inteligentes en la automatización de la documentación SRS [20].

Aboukadri et al. [21] abordan la problemática de la ineficiencia y los errores inherentes a la identificación manual de políticas de control de acceso en entornos ágiles, donde la seguridad suele ser relegada. El objetivo de su estudio fue automatizar la extracción y organización de estas políticas a partir de historias de usuario mediante un marco de trabajo que integra Generación Aumentada por Recuperación (RAG) y Modelos de Lenguaje de Gran Tamaño (LLM). La metodología empleada consistió en un pipeline de procesamiento que utiliza el modelo `gpt2-distilroberta-v1` para generar representaciones vectoriales y almacenar fragmentos de texto en bases de datos semánticas para su posterior recuperación contextual. Pruebas realizadas sobre 22 conjuntos de datos con más de 1,600 historias de usuario demostraron precisiones de hasta el 84.21 %, evidenciando la capacidad de la IA para capturar reglas de acceso complejas de manera escalable [21]. Asimismo, el estudio concluye que el uso de RAG mitiga las alucinaciones y proporciona trazabilidad, aunque reconoce una dependencia crítica de la calidad del texto de entrada. Por otra parte, este antecedente sustenta la presente tesis al validar técnicamente el uso de agentes inteligentes y técnicas de recuperación de contexto para transformar requisitos informales en especificaciones funcionales estructuradas [21].

Zhang et al. [22] abordaron la problemática de la inconsistencia y la ambigüedad en la redacción de historias de usuario, factores que comprometen la velocidad del desarrollo y el cumplimiento de las expectativas del cliente en marcos ágiles. El estudio se propuso evaluar la viabilidad de un sistema de agentes autónomos basado en modelos de lenguaje de gran tamaño (LLM), denominado ALAS, para automatizar la mejora de la calidad de estos requisitos funcionales. La metodología implementada consistió en una arquitectura multi-agente donde perfiles simulados de dueño de producto (PO) e ingeniero de requisitos (RE) interactúan de forma iterativa para refinar descripciones y criterios de aceptación bajo el marco INVEST. Pruebas realizadas en la organización Austrian Post Group IT utilizando modelos GPT-3.5 y GPT-4 revelaron que la intervención de la IA mejora notablemente la claridad y la especificidad del valor de negocio, aunque se observó que modelos más avanzados como GPT-4 tienden a producir requisitos excesivamente largos y complejos [22]. Asimismo, el análisis crítico concluyó que la supervisión humana sigue siendo indispensable para monitorear la integridad de los resultados y mitigar posibles alucinaciones. Por otra parte, esta investigación sustenta la presente tesis al validar em-

píricamente que la colaboración entre agentes especializados es una estrategia eficaz para la transformación y el refinamiento automático de especificaciones funcionales bajo estándares de calidad industrial [22].

Hou et al. [23] abordaron la problemática de la fragmentación tecnológica en la integración de herramientas de inteligencia artificial, la cual genera sistemas rígidos y difíciles de escalar debido a la dependencia de conexiones manuales de APIs. El objetivo central del estudio fue sistematizar el análisis del Model Context Protocol (MCP) para evaluar su arquitectura, ciclo de vida y los riesgos de seguridad emergentes en entornos de agentes autónomos [23]. La metodología empleada consistió en la formalización de dieciséis actividades clave del protocolo y la validación empírica de amenazas mediante el desarrollo de servidores de prueba de concepto en entornos aislados [23]. Los resultados principales revelaron que el MCP logra unificar la comunicación bidireccional y el descubrimiento dinámico de recursos, aunque expone vulnerabilidades críticas como la inyección indirecta de instrucciones y el envenenamiento de herramientas [23]. Asimismo, se concluye que la madurez de este ecosistema requiere protocolos de validación de identidad y mecanismos de aislamiento más estrictos para garantizar la integridad de las operaciones [23]. De igual manera, este trabajo sustenta la presente investigación al proporcionar la base técnica del protocolo MCP, facilitando que una arquitectura multi-agente recupere contextos externos de forma estandarizada para la generación automática de requisitos funcionales de alta precisión [23].

2.2 Ingeniería de Requisitos

La Ingeniería de Requisitos es una disciplina de la Ingeniería de Software encargada de identificar, analizar, documentar, validar y gestionar las necesidades y restricciones que debe satisfacer un sistema software. Su objetivo principal es asegurar que las expectativas de los interesados sean comprendidas y representadas de manera clara, completa y verificable antes de iniciar las etapas de diseño e implementación [1].

En los últimos años, el interés por automatizar algunas de estas tareas mediante técnicas de

procesamiento del lenguaje natural e inteligencia artificial ha incrementado considerablemente, debido a la necesidad de mejorar la calidad de las especificaciones y disminuir el esfuerzo asociado a su elaboración manual [24].

2.2.1 Metodología de Elicitación de Requisitos

La elicitación de requisitos es una de las actividades fundamentales de la Ingeniería de Requisitos, cuyo propósito es identificar, comprender y recopilar las necesidades, expectativas y restricciones de los interesados para definir las funcionalidades y características que deberá satisfacer el sistema software. Una adecuada ejecución de este proceso contribuye a disminuir errores derivados de requisitos incompletos, ambiguos o inconsistentes, reduciendo el riesgo de retrabajos y fallos durante las etapas posteriores del desarrollo [7], [5].

El proceso de elicitación generalmente inicia con la identificación de los interesados y continúa con la selección de técnicas apropiadas para obtener información del dominio del problema. Entre las técnicas más empleadas se encuentran las entrevistas, cuestionarios, talleres colaborativos, observación directa, análisis documental y prototipado. La elección de una técnica específica depende de factores como la complejidad del sistema, la disponibilidad de los participantes y el contexto organizacional en el que se desarrolla el proyecto [7], [25]. En metodologías ágiles, la elicitación se considera una actividad iterativa que permite refinar continuamente los requisitos conforme evolucionan las necesidades del negocio.

En los últimos años, la incorporación de técnicas de inteligencia artificial y procesamiento del lenguaje natural ha permitido asistir el proceso de elicitación mediante la extracción automática de información, la identificación de requisitos implícitos y el análisis de conversaciones sostenidas con los interesados. Estas tecnologías buscan complementar las prácticas tradicionales y mejorar la eficiencia en la generación de especificaciones de requisitos, manteniendo la participación humana como elemento esencial para validar la información obtenida y garantizar su correspondencia con las necesidades reales de los usuarios [5].

2.2.2 Estándar IEEE 830-1998

El estándar IEEE 830-1998 establece una práctica recomendada para la elaboración de documentos de Especificación de Requisitos de Software (SRS), proporcionando una estructura organizada para describir de manera clara, completa y verificable las funcionalidades, restricciones y características de un sistema software. Entre los atributos de calidad que deben cumplir los requisitos especificados se encuentran la ausencia de ambigüedad, consistencia, completitud, trazabilidad y verificabilidad, con el propósito de facilitar la comunicación entre clientes, usuarios y desarrolladores durante el ciclo de vida del proyecto [5], [1].

Aunque el estándar fue reemplazado posteriormente por la norma ISO/IEC/IEEE 29148, el IEEE 830-1998 continúa siendo ampliamente utilizado en entornos académicos y de investigación debido a la simplicidad de su estructura y a su adopción en numerosos trabajos relacionados con la automatización de documentos SRS. La especificación propuesta por este estándar se organiza en secciones que incluyen la introducción, descripción general del sistema, requisitos específicos, interfaces externas, requisitos funcionales, restricciones de diseño y otros atributos de calidad relevantes para el sistema desarrollado [7].

Si bien la norma ISO/IEC/IEEE 29148 ha sustituido oficialmente al IEEE 830-1998, este último sigue teniendo vigencia en el ámbito académico. Su estructura clara y su extensa utilización en estudios enfocados en la creación y valoración automática de documentos de SRS explican que aún sea ampliamente empleado en entornos de investigación.

2.2.3 Tipos de requisitos: funcionales

Los requisitos de software representan las capacidades, restricciones y condiciones que debe cumplir un sistema para satisfacer las necesidades de sus usuarios y demás partes interesadas. De manera general, los requisitos suelen clasificarse en funcionales y no funcionales. Los requisitos funcionales describen los servicios, comportamientos y funciones específicas que el sistema debe proporcionar, mientras que los requisitos no funcionales establecen atributos de

calidad relacionados con el rendimiento, seguridad, usabilidad, confiabilidad y mantenibilidad del software [1], [5].

Los requisitos funcionales constituyen uno de los elementos más importantes dentro de un documento de SRS, debido a que definen las operaciones que deberá ejecutar el sistema en respuesta a determinadas entradas, condiciones o interacciones con los usuarios. La adecuada identificación y documentación de estos requisitos facilita las actividades posteriores de diseño, implementación y pruebas, además de contribuir a mejorar la trazabilidad y verificabilidad del producto desarrollado [7]. Diversos estudios recientes han explorado el uso de técnicas de procesamiento de lenguaje natural y modelos de lenguaje de gran escala para automatizar la extracción y clasificación de requisitos funcionales a partir de entrevistas, historias de usuario y otros artefactos expresados en lenguaje natural [2].

En el contexto de esta investigación, el sistema multi-agente propuesto se enfoca principalmente en la identificación y clasificación automática de requisitos funcionales obtenidos a partir de audios de entrevistas realizadas a Product Owners. Posteriormente, estos requisitos son enriquecidos contextualmente y organizados en un documento SRS estructurado conforme al estándar IEEE 830-1998, permitiendo disminuir el esfuerzo asociado a la elaboración manual de especificaciones y mejorar la calidad de la documentación generada [5], [6].

2.3 Inteligencia Artificial aplicada a la Ingeniería de Software

La Inteligencia Artificial (IA) aplicada a la Ingeniería de Software comprende el uso de técnicas y modelos computacionales capaces de asistir, automatizar o mejorar las diferentes actividades del ciclo de vida del desarrollo de software. En los últimos años, el avance de los modelos de aprendizaje profundo y, especialmente, de los Modelos de Lenguaje de Gran Escala (LLMs), ha impulsado el desarrollo de herramientas capaces de apoyar tareas como la generación de código, la documentación, el análisis de requisitos, la detección de defectos y la asistencia a desarrolladores, incrementando la productividad y reduciendo el esfuerzo asociado a actividades tradicionalmente manuales [5], [1].

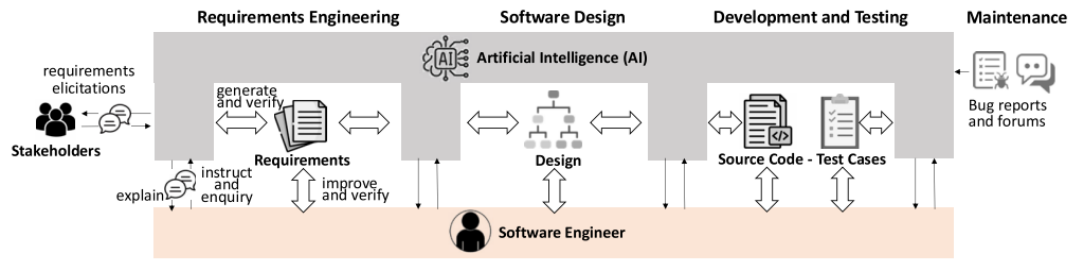


Figura 3.

Arquitectura lógica de la simbiosis futura entre ingenieros de software e IA[16].

La figura 3 plantean una visión de simbiosis futura entre ingenieros de software e IA que abarca las principales fases del ciclo de vida del desarrollo: Ingeniería de Requisitos, Diseño de Software, Desarrollo y Pruebas, y Mantenimiento, tal como se ilustra en la Figura 3. Según los autores, en la fase de requisitos los agentes de IA podrán asistir en la elicitación mediante conversaciones con los interesados, resumir documentación extensa y traducirla en especificaciones formales a partir de necesidades expresadas en lenguaje natural. Esta visión respalda el enfoque de la presente investigación, la cual se centra específicamente en automatizar dicha fase mediante un sistema multi-agente basado en MCP.

2.3.1 Modelos de Lenguaje de Gran Escala (LLMs)

Los Modelos de Lenguaje de Gran Escala (Large Language Models, LLMs) son sistemas de inteligencia artificial entrenados sobre grandes volúmenes de datos textuales con el objetivo de comprender, generar y transformar lenguaje natural. Estos modelos se basan principalmente en la arquitectura Transformer, la cual emplea mecanismos de atención para capturar relaciones contextuales entre palabras y secuencias de texto, permitiendo realizar tareas como generación de contenido, resumen automático, clasificación de información, traducción y asistencia en el desarrollo de software [26], [27].

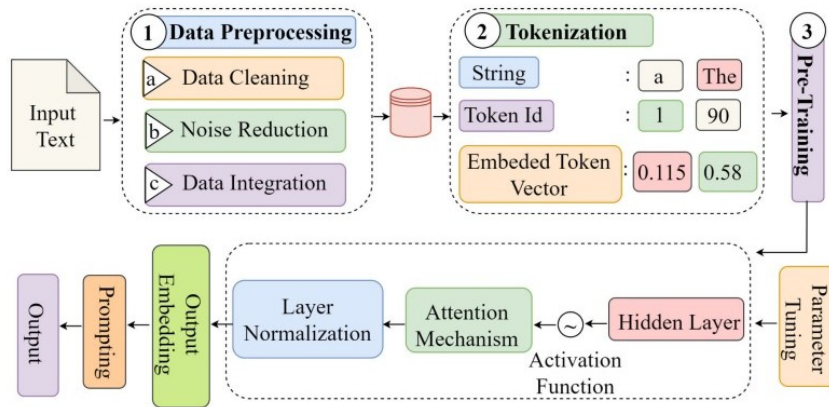


Figura 4.

Proceso general de funcionamiento de un LLM [28].

Raiaan et al. [28] describen el proceso general de funcionamiento de un LLM como una secuencia de etapas que inicia con el preprocesamiento de los datos de entrada (limpieza, reducción de ruido e integración de datos), seguido de la tokenización, en la que el texto se convierte en identificadores numéricos y vectores de embedding. Posteriormente, durante la fase de pre-entrenamiento, el modelo ajusta sus parámetros mediante mecanismos de atención y normalización de capas, para finalmente generar una representación de salida (*output embedding*) que es empleada en la etapa de *prompting* para producir la respuesta final, tal como se ilustra en la Figura 4.

El desempeño de los LLMs ha impulsado su adopción en diferentes áreas de la Ingeniería de Software, incluyendo la Ingeniería de Requisitos, donde se utilizan para apoyar actividades de elicitación, análisis, validación y generación de especificaciones. Investigaciones recientes evidencian que estos modelos pueden contribuir a mejorar la calidad de documentos de requisitos, identificar inconsistencias y automatizar tareas tradicionalmente realizadas por expertos humanos. Sin embargo, la mayoría de aplicaciones reportadas se desarrollan en entornos controlados y aún presentan limitaciones relacionadas con la precisión de las respuestas y la comprensión del contexto específico del dominio [5], [29].

Los LLMs constituyen el núcleo de procesamiento de los agentes inteligentes encargados de

analizar transcripciones de entrevistas, clasificar requisitos funcionales y generar documentos de SRS conforme al estándar IEEE 830-1998. Su capacidad para interpretar lenguaje natural y producir texto estructurado permite automatizar actividades de Ingeniería de Requisitos que tradicionalmente requieren una elevada intervención manual [5], [30].

2.3.2 Limitaciones de los LLMs

A pesar de sus capacidades para comprender y generar texto en lenguaje natural, los Modelos de LLMs presentan diversas limitaciones que restringen su aplicación en dominios que requieren alta precisión, como la Ingeniería de Requisitos. Una de las principales problemáticas es la generación de información incorrecta o inexistente, fenómeno conocido como *hallucination* o alucinación, en el que el modelo produce respuestas plausibles pero sin sustento en fuentes verificables. Asimismo, estos modelos pueden heredar sesgos presentes en los datos de entrenamiento, afectando la objetividad y confiabilidad de los resultados generados [31], [32].

Otra limitación importante corresponde a la gestión de contextos extensos y especializados. Aunque los LLMs modernos permiten procesar grandes cantidades de información, su desempeño puede degradarse cuando deben analizar documentos largos, múltiples conversaciones o información altamente específica de un dominio determinado. Adicionalmente, el conocimiento incorporado en el modelo se encuentra limitado a la fecha de corte de sus datos de entrenamiento, impidiendo acceder a información actualizada o a fuentes externas en tiempo real [9], [27].

Estas limitaciones resultan especialmente relevantes en tareas de Ingeniería de Requisitos, donde es necesario preservar la consistencia, completitud y trazabilidad de las especificaciones generadas. Por esta razón, investigaciones recientes han propuesto complementar los LLMs mediante técnicas de recuperación de información, arquitecturas multi-agente y mecanismos de inyección de contexto externo, con el propósito de mejorar la calidad de las respuestas y reducir la probabilidad de generar información errónea. En este trabajo de investigación, dichas limitaciones motivan la incorporación de un agente de enriquecimiento contextual y el uso del Model Context Protocol (MCP) para proporcionar información adicional a los modelos de lenguaje

empleados [1], [5].

2.3.3 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) es una técnica que combina modelos de lenguaje de gran escala con mecanismos de recuperación de información externa para mejorar la precisión y actualidad de las respuestas generadas. A diferencia de los LLMs convencionales, que dependen únicamente del conocimiento adquirido durante su entrenamiento, RAG incorpora información relevante obtenida dinámicamente desde bases de datos, repositorios documentales o fuentes externas, permitiendo proporcionar respuestas fundamentadas en evidencia y reduciendo la probabilidad de generar información incorrecta o desactualizada [33], [34].

El funcionamiento de RAG se basa generalmente en dos etapas principales: recuperación y generación. En la primera etapa, un sistema de búsqueda identifica y selecciona documentos o fragmentos de texto relacionados con la consulta del usuario mediante técnicas de similitud semántica o búsqueda vectorial. Posteriormente, la información recuperada es incorporada al contexto de entrada del modelo de lenguaje, el cual utiliza dichos datos para generar una respuesta más precisa y contextualizada. Este enfoque ha demostrado mejoras significativas en tareas de respuesta a preguntas, resumen automático, generación de código y aplicaciones especializadas en dominios como la medicina, educación e ingeniería de software [33], [35].

En el ámbito de la Ingeniería de Requisitos, RAG puede emplearse para enriquecer especificaciones obtenidas durante entrevistas de elicitación, proporcionando información complementaria proveniente de estándares, documentación técnica o fuentes de conocimiento especializadas. En esta investigación, el Agente de Enriquecimiento Contextual implementa un enfoque basado en RAG para complementar los requisitos funcionales identificados a partir de las transcripciones de entrevistas, utilizando información obtenida mediante servicios externos accesibles a través del MCP, con el propósito de mejorar la completitud y coherencia del documento SRS generado automáticamente [34], [27].

2.4 Sistemas Multi-agente

Los sistemas multiagente (MAS) son plataformas distribuidas compuestas por varios agentes autónomos que se comunican entre ellos y con el ambiente que los rodea, con el fin de dar solución a problemas complejos de forma colaborativa. Al repartir las tareas entre agentes con funciones específicas, se facilita el manejo de problemas que serían complejos de resolver con un solo elemento inteligente, lo que incrementa la capacidad de crecimiento, la adaptabilidad y la solidez del sistema.

2.4.1 Definición y características de los agentes de IA

Un agente de inteligencia artificial es una entidad de software capaz de percibir información de su entorno, procesarla, tomar decisiones y ejecutar acciones con el propósito de alcanzar objetivos específicos. A diferencia de los programas tradicionales, los agentes poseen cierto grado de autonomía y pueden adaptarse dinámicamente a cambios en el entorno, utilizando mecanismos de razonamiento, memoria y aprendizaje para mejorar su desempeño [9], [36].

Entre las principales características de los agentes de IA se encuentran la autonomía, que les permite operar sin intervención humana constante; la reactividad, que facilita responder a eventos del entorno; la proactividad, relacionada con la capacidad de anticiparse a situaciones futuras; y la habilidad social, que posibilita la interacción y cooperación con otros agentes o usuarios. En sistemas basados en LLMs, estas capacidades pueden complementarse mediante el uso de herramientas externas, memoria persistente y acceso a fuentes de conocimiento especializadas [36], [37].

En esta investigación, cada agente especializado desempeña una función específica dentro del proceso de generación automática de documentos SRS. El agente de transcripción convierte el audio en texto, el agente de análisis identifica y clasifica requisitos funcionales, el agente de enriquecimiento contextual recupera información adicional mediante RAG y el agente generador construye el documento SRS conforme al estándar IEEE 830-1998. La especialización de tareas

permite reducir la complejidad del sistema y mejorar la calidad de los resultados obtenidos [9], [38].

2.4.2 Arquitecturas multi-agente basadas en LLMs

Las arquitecturas multi-agente basadas en Modelos de Lenguaje de Gran Escala están compuestas por múltiples agentes inteligentes que colaboran para resolver tareas complejas mediante la distribución de responsabilidades y el intercambio de información. Cada agente puede estar especializado en actividades específicas, tales como planificación, razonamiento, recuperación de conocimiento, uso de herramientas externas o generación de contenido, permitiendo superar las limitaciones de los LLMs individuales [9], [38].

Diversos estudios han propuesto arquitecturas organizadas de forma jerárquica, colaborativa o basada en roles. En las arquitecturas jerárquicas, un agente coordinador supervisa la ejecución de agentes subordinados, mientras que en las arquitecturas colaborativas los agentes intercambian información de manera descentralizada para alcanzar un objetivo común. También existen enfoques basados en roles, donde cada agente posee responsabilidades claramente definidas dentro del flujo de trabajo del sistema [9], [37].

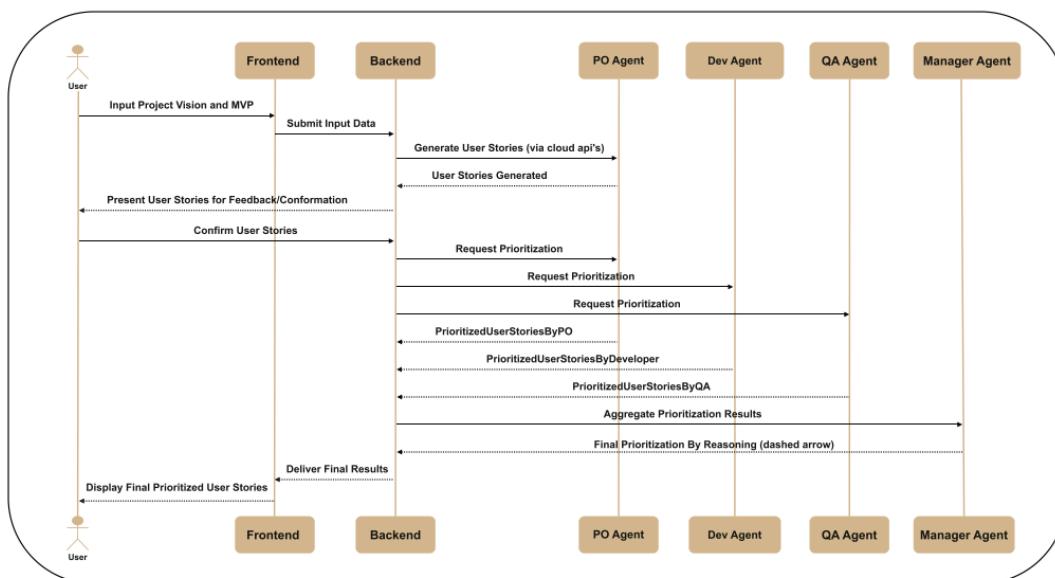


Figura 5.

Ejemplo de arquitectura multi-agente basada en roles para generación y priorización de historias de usuario [15].

La Figura 5 ilustra un ejemplo de arquitectura multi-agente basada en roles, en la que agentes especializados (Product Owner, Desarrollador, QA y Gestor) colaboran para generar y priorizar historias de usuario a partir de la visión del proyecto proporcionada por el usuario. En este esquema, un agente Manager centraliza las priorizaciones individuales emitidas por cada agente especializado y determina, mediante razonamiento, la priorización final que se entrega al usuario. Este tipo de coordinación evidencia cómo la especialización de roles y la comunicación estructurada entre agentes permiten resolver tareas colaborativas que un único modelo de lenguaje no podría abordar de forma aislada [15].

2.4.3 Coordinación y comunicación entre agentes

La coordinación y comunicación entre agentes constituyen elementos fundamentales en los sistemas multi-agente, ya que permiten organizar la ejecución de tareas, intercambiar información y alcanzar objetivos comunes de manera eficiente. En arquitecturas basadas en inteligencia artificial generativa, la coordinación puede realizarse mediante agentes supervisores u orquestadores encargados de asignar responsabilidades, controlar el flujo de trabajo y gestionar la interacción entre agentes especializados. Este enfoque contribuye a mejorar la modularidad, escalabilidad y capacidad de adaptación del sistema ante cambios en los requerimientos o en el entorno de operación [9], [38].

Los mecanismos de comunicación entre agentes pueden implementarse utilizando intercambio de mensajes, memoria compartida, bases de conocimiento comunes o protocolos estandarizados que faciliten la interoperabilidad entre componentes heterogéneos. En sistemas multi-agente basados en LLMs, la comunicación estructurada permite transmitir instrucciones, resultados parciales y contexto adicional entre agentes, reduciendo la pérdida de información y favoreciendo procesos colaborativos de razonamiento. Asimismo, la incorporación de herramientas externas y servicios especializados amplía las capacidades de los agentes, permitiéndoles acceder a información actualizada y ejecutar acciones fuera de las capacidades intrínsecas del modelo de lenguaje [9], [37].

En esta investigación, la coordinación del sistema es realizada por un Agente Orquestador que dirige la ejecución secuencial de los agentes de transcripción, análisis y clasificación, enriquecimiento contextual y generación del documento SRS. La comunicación entre estos agentes y los servicios externos se gestiona mediante el MCP, el cual proporciona una interfaz estandarizada para intercambiar contexto, invocar herramientas y acceder a recursos externos, garantizando un flujo de información estructurado durante el proceso de generación automatizada del documento de especificación de requisitos [38], [36].

2.4.4 Frameworks para sistemas multi-agente

Los frameworks para sistemas multi-agente proporcionan componentes reutilizables y mecanismos de abstracción que facilitan el desarrollo, coordinación y ejecución de agentes inteligentes. En el contexto de aplicaciones basadas en Modelos de Lenguaje de Gran Escala (LLMs), estos frameworks permiten definir roles especializados, administrar estados conversacionales, integrar herramientas externas y gestionar flujos de trabajo complejos, reduciendo el esfuerzo de implementación y mejorando la mantenibilidad de las soluciones desarrolladas [9], [38].

Entre los frameworks más utilizados se encuentran AutoGen, LangGraph, CrewAI y Semantic Kernel. AutoGen permite construir conversaciones entre múltiples agentes autónomos capaces de colaborar para resolver tareas complejas, mientras que LangGraph facilita la definición de flujos de trabajo mediante grafos dirigidos y estados persistentes. Por su parte, CrewAI se orienta a la creación de equipos de agentes con roles específicos y Semantic Kernel proporciona mecanismos para integrar modelos de lenguaje con herramientas, memoria y servicios externos [38], [39]. Estas plataformas han ganado relevancia debido a su flexibilidad para implementar arquitecturas multi-agente en aplicaciones de automatización, asistencia inteligente e ingeniería de software.

En esta investigación, aunque el sistema propuesto no depende de un framework multi-agente específico, las capacidades ofrecidas por estas herramientas constituyen una referencia para el diseño de la arquitectura planteada. En particular, la existencia de un agente orquestador,

agentes especializados y mecanismos de comunicación con servicios externos guarda similitud con las estrategias de coordinación implementadas en dichos frameworks, facilitando además una posible evolución futura del prototipo hacia soluciones más robustas y escalables [9].

2.5 Model Context Protocol (MCP)

El Protocolo de Contexto para Modelos (MCP, por sus siglas en inglés) se presenta como un estándar pensado para simplificar la comunicación entre modelos de lenguaje de gran escala y elementos externos, brindando un acceso ordenado a herramientas, bases de datos y servicios especializados. Su finalidad fundamental es potenciar las habilidades de los sistemas de inteligencia artificial al integrar información contextual en tiempo real y facilitar la conexión entre distintas aplicaciones.[40]

2.5.1 Definición y propósito del MCP

El MCP es un estándar abierto que define una forma uniforme de vincular modelos lingüísticos con herramientas, repositorios de datos y servicios externos. Su meta es brindar a los sistemas de inteligencia artificial un acceso regulado a información complementaria y a funciones que van más allá de su conocimiento base, lo que les permite potenciar su capacidad de razonamiento y la ejecución de tareas complejas. Con ello, el protocolo pretende dar respuesta a inconvenientes como la carencia de datos recientes, la excesiva dependencia del conjunto de entrenamiento y las complicaciones para articular diversos servicios en una sola plataforma.

La arquitectura propuesta integra el MCP para operacionalizar un enfoque inspirado en arquitecturas orientadas a servicios, bajo el cual los agentes inteligentes pueden solicitar información o ejecutar acciones a través de servidores especializados que exponen recursos y herramientas específicas. Esta estrategia técnica se fundamenta en la capacidad de uso de herramientas (tool-use), la cual permite a los modelos de lenguaje aprovechar recursos externos para ampliar sus funciones y operar de manera más efectiva en entornos diversos. Este diseño modu-

lar favorece la interoperabilidad entre aplicaciones heterogéneas y maximiza la reutilización de componentes, permitiendo incorporar nuevas capacidades al sistema sin necesidad de modificar o reentrenar el modelo de lenguaje subyacente [9]

En esta investigación, se implementa el MCP como el mecanismo de comunicación para conectar los agentes especializados con servicios externos de búsqueda web, almacenamiento de documentos y generación del documento SRS. Su utilización permite operacionalizar la inyección de contexto adicional a los agentes inteligentes, mejorando la calidad de la información procesada y mitigando el riesgo de alucinaciones. Asimismo, este enfoque garantiza una arquitectura modular que facilita la gestión del sistema y futuras ampliaciones del prototipo desarrollado [9], [38].

2.5.2 Arquitectura cliente-servidor

El MCP se basa en un modelo arquitectónico cliente-servidor que posibilita una interacción estandarizada entre aplicaciones impulsadas por modelos de lenguaje y servicios externos. Bajo este enfoque, el cliente MCP se encarga de originar las peticiones y de coordinar la comunicación con uno o múltiples servidores MCP, mientras que estos últimos ponen a disposición recursos, herramientas y capacidades que el modelo de lenguaje puede aprovechar al momento de ejecutar una tarea concreta. Esta separación de responsabilidades facilita la modularidad del sistema y permite incorporar nuevas capacidades sin modificar la lógica interna del modelo de inteligencia artificial.

La arquitectura propuesta utiliza el MCP como mecanismo para establecer una comunicación estructurada entre clientes y servidores mediante el intercambio de mensajes en formatos estandarizados. Su implementación permite descubrir capacidades disponibles en servidores especializados y conectar a un mismo cliente con múltiples fuentes de información, tales como motores de búsqueda, bases de datos o sistemas de archivos. Esta estrategia técnica se fundamenta en la capacidad de uso de herramientas (tool-use), la cual favorece la interoperabilidad entre componentes heterogéneos y simplifica la integración de recursos externos dentro de apli-

caciones basadas en Modelos de Lenguaje de Gran Escala [9].

En la presente investigación, el Agente Orquestador desempeña el rol de cliente MCP, coordinando la comunicación con diversos servidores especializados. Entre ellos se encuentran un servidor de búsqueda web para el enriquecimiento contextual de requisitos, un servidor de base de datos encargado del almacenamiento de documentos SRS generados y un servidor orientado al soporte de generación estructurada del documento final. Esta arquitectura permite desacoplar los servicios externos de los procesos cognitivos de los agentes, siguiendo los principios de modularidad y orquestación propuestos por [9], lo que facilita la escalabilidad y especialización necesaria para tareas de ingeniería de requisitos complejas [38].

2.5.3 Resources, Tools y Prompts

Los conceptos de Resources, Tools y Prompts constituyen los elementos fundamentales del MCP, permitiendo estandarizar la interacción entre modelos de lenguaje y servicios externos. Los Resources representan fuentes de información que pueden ser consultadas por un modelo, tales como documentos, archivos, bases de datos o contenido web. Los Tools corresponden a funciones ejecutables expuestas por un servidor MCP que permiten al modelo interactuar con el entorno mediante acciones que generan efectos secundarios, tales como efectuar búsquedas, consultar repositorios, modificar bases de datos o invocar APIs externas. Finalmente, los Prompts actúan como plantillas reutilizables que estructuran la interacción, permitiendo una orquestación de flujos de trabajo más eficiente [23].

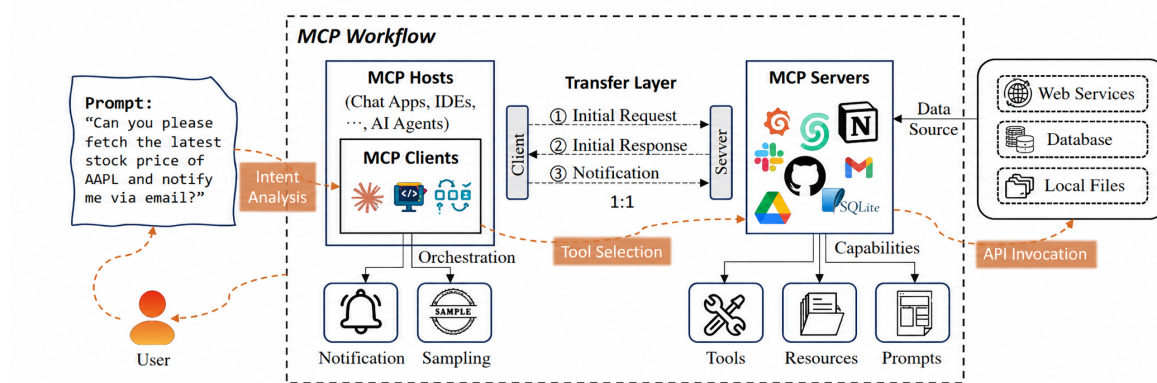


Figura 6.
Flujo de trabajo del Model Context Protocol [23].

La Figura 6 ilustra el flujo de trabajo completo del MCP, desde que el usuario formula una solicitud en lenguaje natural hasta la ejecución de las capacidades correspondientes. El MCP Host aloja al cliente MCP, el cual envía una solicitud inicial al servidor a través de la capa de transferencia, estableciendo una comunicación de tipo 1:1 entre cliente y servidor. Una vez seleccionada la herramienta apropiada mediante el proceso de *Tool Selection*, el servidor expone sus capacidades organizadas en las tres categorías mencionadas: Tools, Resources y Prompts, las cuales son invocadas para satisfacer la intención detectada en la solicitud del usuario [23].

La separación entre recursos, herramientas y prompts permite desacoplar el contexto informativo, las capacidades operativas y las estructuras de interacción de un sistema basado en LLMs. Esta organización favorece el descubrimiento dinámico de capacidades en servidores MCP y mejora la interoperabilidad en entornos heterogéneos. Investigaciones recientes como ScaleMCP demuestran que esta arquitectura, combinada con mecanismos de auto-sincronización, permite a los agentes gestionar de forma autónoma un vasto ecosistema de herramientas, superando las limitaciones de conocimiento estático y proporcionando acceso a datos en tiempo real fuera del alcance interno del modelo [23], [41].

En el contexto del presente trabajo, los servidores MCP implementados exponen principalmente herramientas (Tools) para la búsqueda de información y almacenamiento de documentos, mientras que los Prompts son utilizados internamente por los agentes para estructurar la generación del documento SRS.

2.5.4 Integración del MCP con agentes de IA

La integración del MCP con agentes de inteligencia artificial permite ampliar las capacidades de los modelos de lenguaje mediante el acceso estandarizado a recursos, herramientas y servicios externos. En lugar de depender únicamente del conocimiento adquirido durante el entrenamiento, los agentes pueden utilizar MCP para consultar información actualizada, ejecutar acciones específicas y compartir contexto con otros componentes del sistema. Este enfoque favorece el desarrollo de agentes más autónomos, adaptables y capaces de resolver tareas complejas que requieren interacción con múltiples fuentes de información [23].

En arquitecturas multi-agente basadas en LLMs, MCP actúa como una capa de interoperabilidad que facilita la comunicación entre agentes especializados y servidores externos. A través de este protocolo, los agentes pueden descubrir dinámicamente las capacidades disponibles, invocar herramientas remotas y acceder a recursos compartidos sin necesidad de implementar integraciones específicas para cada servicio. Investigaciones recientes destacan que esta capacidad contribuye a mejorar la modularidad, escalabilidad y mantenibilidad de sistemas inteligentes compuestos por múltiples agentes colaborativos [23], [41].

En la presente investigación, el Agente Orquestador desempeña el rol de cliente MCP, coordinando la comunicación con diversos servidores especializados bajo el estándar abierto introducido por [40]. Entre estos servidores se incluyen uno de búsqueda web para el enriquecimiento contextual de requisitos, uno de base de datos para el almacenamiento de documentos SRS y un servidor orientado al soporte de generación estructurada. Esta arquitectura permite desacoplar los servicios externos de los procesos cognitivos de los agentes, facilitando la escalabilidad y el mantenimiento del sistema multi-agente propuesto, en línea con los principios de orquestación y especialización requeridos para tareas de ingeniería de requisitos de alta complejidad [9], [38].

2.6 Reconocimiento Automático de Voz (ASR)

El Reconocimiento Automático de Voz (Automatic Speech Recognition, ASR) es una tecnología de inteligencia artificial que permite convertir señales de audio habladas en texto legible por máquinas. Gracias a los avances recientes en aprendizaje profundo y modelos neuronales de gran escala, los sistemas ASR han alcanzado niveles de precisión que posibilitan su aplicación en diversos dominios, incluyendo asistentes virtuales, sistemas de transcripción, atención al cliente e Ingeniería de Software [42], [43]

En el contexto de la Ingeniería de Requisitos, el ASR representa una alternativa prometedora para automatizar la transcripción de entrevistas de elicitación, reduciendo el esfuerzo manual asociado a la documentación y facilitando el procesamiento posterior de la información mediante modelos de lenguaje y sistemas multi-agente [43], [5].

2.6.1 Fundamentos del ASR

El Reconocimiento Automático de Voz (Automatic Speech Recognition, ASR) es una tecnología que permite transformar señales de voz en texto mediante técnicas de procesamiento digital de señales y aprendizaje automático. Los sistemas ASR modernos emplean modelos neuronales profundos capaces de aprender representaciones acústicas y lingüísticas directamente a partir de grandes volúmenes de datos, eliminando la necesidad de diseñar manualmente características específicas del habla. El objetivo principal de estos sistemas es minimizar la tasa de error de palabras (Word Error Rate, WER) y generar transcripciones precisas incluso en presencia de ruido, variaciones de pronunciación o diferencias entre hablantes [43].

El proceso de reconocimiento de voz comprende generalmente varias etapas, incluyendo el preprocesamiento de la señal de audio, la extracción de características acústicas, el modelado acústico, el modelado del lenguaje y la decodificación de la secuencia de palabras más probable. En los últimos años, las arquitecturas basadas en Transformers y modelos de aprendizaje auto-supervisado han demostrado mejoras significativas respecto a enfoques tradicionales basados en

modelos ocultos de Markov y redes neuronales recurrentes, permitiendo obtener sistemas más robustos y capaces de generalizar a diferentes idiomas y dominios de aplicación [44].

En el contexto de esta investigación, el ASR constituye el primer componente del sistema multi-agente propuesto, siendo responsable de convertir los audios de entrevistas realizadas a Product Owners en transcripciones textuales que posteriormente serán procesadas por agentes especializados en análisis, enriquecimiento contextual y generación automática del documento SRS. La precisión alcanzada durante esta etapa resulta fundamental, ya que errores en la transcripción pueden propagarse al resto del flujo de procesamiento y afectar la calidad de las especificaciones generadas [43], [5].

2.6.2 Modelos open source para transcripción

Los avances recientes en reconocimiento automático de voz han impulsado el desarrollo de diversos modelos de código abierto capaces de realizar transcripciones con altos niveles de precisión. Entre los más destacados se encuentra Whisper, desarrollado por OpenAI, el cual fue entrenado mediante aprendizaje débilmente supervisado utilizando aproximadamente 680 000 horas de datos de audio multilingües recopilados de Internet. Gracias a esta estrategia, Whisper presenta una elevada robustez frente a ruido de fondo, acentos y variaciones lingüísticas. Esto le permite generalizar con éxito en un entorno zero-shot, logrando resultados competitivos en diversos dominios sin necesidad de un ajuste fino (fine-tuning) específico para cada conjunto de datos [43].

Otro modelo ampliamente utilizado es wav2vec 2.0, propuesto por Meta AI, que emplea aprendizaje autosupervisado para obtener representaciones del habla a partir de grandes cantidades de audio sin etiquetar [44]. Tras un proceso de ajuste fino con datos anotados, este modelo logra desempeños competitivos en diferentes conjuntos de evaluación, destacando por su eficiencia en entornos de bajos recursos de datos [44], [43]. Asimismo, DeepSpeech constituye una alternativa de código abierto basada en redes neuronales profundas, cuya arquitectura ha facilitado el despliegue de sistemas de reconocimiento de voz robustos en diversos dominios

operativos [43].

El uso de modelos open source para transcripción resulta adecuado debido a su flexibilidad, disponibilidad y capacidad para integrarse dentro de arquitecturas multi-agente. Particularmente, Whisper representa una opción prometedora para el Agente de Transcripción ASR, ya que ofrece soporte multilingüe, alta precisión en escenarios reales y puede ejecutarse localmente o mediante servicios externos, facilitando su incorporación al prototipo propuesto para procesar entrevistas de elicitación. En este flujo, el ASR permite transformar el lenguaje natural de los interesados en datos procesables por el sistema multi-agente, optimizando la captura de necesidades técnicas [43], [7].

2.6.3 ASR aplicado a entrevistas de elicitación

Las entrevistas constituyen una de las técnicas de elicitación de requisitos más utilizadas en Ingeniería de Requisitos, ya que permiten obtener información detallada sobre las necesidades, expectativas y restricciones de los interesados. Sin embargo, la documentación manual de estas sesiones suele ser una actividad tediosa y propensa a errores, especialmente cuando las entrevistas poseen una larga duración o involucran múltiples participantes. En este contexto, los sistemas de Reconocimiento Automático de Voz (ASR) ofrecen una alternativa para automatizar la transcripción de las conversaciones, facilitando su posterior análisis y procesamiento [43], [5].

La incorporación de tecnologías ASR en procesos de elicitación permite disponer de registros textuales completos de las entrevistas, reduciendo la pérdida de información y mejorando la trazabilidad entre las necesidades expresadas por los usuarios y los requisitos documentados. Asimismo, las transcripciones generadas pueden ser procesadas mediante modelos de lenguaje de gran escala para identificar requisitos funcionales, detectar inconsistencias y asistir en la construcción de documentos de especificación. Estudios recientes evidencian un creciente interés por combinar técnicas de procesamiento del lenguaje natural e inteligencia artificial para brindar soporte automatizado a actividades de Ingeniería de Requisitos [5], [1].

El ASR representa el punto de partida del flujo de procesamiento del sistema multi-agente propuesto. El agente de transcripción recibe el audio de entrevistas realizadas a Product Owners y genera una representación textual que posteriormente es utilizada por agentes especializados en análisis, enriquecimiento contextual y generación del documento SRS conforme al estándar IEEE 830-1998. De esta manera, la automatización de la transcripción contribuye a disminuir el esfuerzo asociado a la documentación manual y proporciona una base estructurada para las etapas posteriores del proceso [43], [1].

2.6.4 Integración del ASR en sistemas multi-agente

La integración del Reconocimiento Automático de Voz (ASR) en sistemas multi-agente permite distribuir las tareas de procesamiento del habla y análisis de información entre componentes especializados, mejorando la modularidad y escalabilidad de la solución desarrollada. En este enfoque, un agente dedicado al reconocimiento de voz es responsable de transformar el audio en texto, mientras que otros agentes pueden encargarse del análisis semántico, recuperación de información, razonamiento o generación de documentos. Esta separación de responsabilidades facilita el mantenimiento del sistema y permite sustituir o actualizar componentes individuales sin afectar el funcionamiento global de la arquitectura [43], [9].

Las arquitecturas multi-agente basadas en modelos de lenguaje han demostrado ser adecuadas para resolver tareas complejas que requieren el procesamiento secuencial de información proveniente de distintas fuentes. La incorporación de un agente ASR como etapa inicial del flujo de trabajo posibilita capturar conocimiento expresado verbalmente y convertirlo en información estructurada que puede ser utilizada por otros agentes especializados. Además, la combinación de tecnologías ASR con modelos de lenguaje de gran escala favorece la automatización de procesos que tradicionalmente dependen de intervención humana, incluyendo actividades relacionadas con la Ingeniería de Requisitos [9], [5].

En la presente investigación, el Agente de Transcripción ASR constituye el primer componente del sistema multi-agente propuesto y tiene la responsabilidad de procesar los audios de

entrevistas realizadas a Product Owners. La salida generada por este agente es enviada al Agente de Análisis y Clasificación, encargado de identificar requisitos funcionales, para posteriormente ser enriquecida mediante técnicas RAG y utilizada en la construcción automática del documento SRS conforme al estándar IEEE 830-1998. Esta integración permite establecer un flujo de procesamiento organizado y favorece la generación de especificaciones de requisitos con menor intervención manual [43], [1].

2.7 Arquitectura e Integración de Aplicaciones Web

El desarrollo de aplicaciones web ha evolucionado significativamente con la adopción de arquitecturas desacopladas y servicios distribuidos, permitiendo construir sistemas más escalables, mantenibles y adaptables a diferentes entornos de ejecución. Estas aplicaciones suelen estar compuestas por componentes especializados que interactúan a través de interfaces bien definidas, facilitando la integración de tecnologías emergentes como modelos de lenguaje, sistemas multi-agente y servicios de inteligencia artificial [45], [46].

En el marco de este estudio, la plataforma web actúa como el canal de comunicación entre el usuario y el sistema multi-agente diseñado, facilitando la carga de grabaciones de entrevistas para la obtención de requisitos, la consulta de los documentos SRS generados y el resguardo de los datos obtenidos. En los apartados siguientes se detallan los fundamentos de las arquitecturas web, las formas de conexión entre el frontend, el backend y los agentes inteligentes, así como las técnicas empleadas para mostrar los resultados y los informes generados.

2.7.1 Arquitectura de aplicaciones web

Las arquitecturas de aplicaciones web definen la organización de los componentes de software y las interacciones necesarias para proporcionar servicios a los usuarios finales. Tradicionalmente, estas arquitecturas se estructuran en capas diferenciadas, donde el frontend se encarga de la interfaz de usuario, el backend procesa la lógica de negocio y las bases de datos gestionan

la persistencia de la información. Esta separación favorece la mantenibilidad, reutilización de componentes y escalabilidad de la aplicación [45], [46].

En aplicaciones modernas, es común adoptar arquitecturas basadas en APIs REST, las cuales permiten establecer mecanismos de comunicación independientes de la tecnología utilizada en cada componente. De esta manera, clientes web, servicios de inteligencia artificial y sistemas externos pueden intercambiar información mediante solicitudes HTTP estandarizadas, facilitando la integración de funcionalidades distribuidas. No obstante, esta estructura requiere una gestión rigurosa de la latencia de red y la implementación de controles de seguridad en cada punto de conexión, dado que la superficie de ataque aumenta en comparación con las arquitecturas monolíticas [46], [45].

En este trabajo, la aplicación web se desarrolló bajo un esquema arquitectónico de capas múltiples, en el que una interfaz gráfica posibilita la carga de archivos de audio provenientes de entrevistas y la revisión de los documentos SRS producidos. El backend funciona como puente entre el usuario y el sistema multi-agente, encargándose de recibir peticiones, orquestar el flujo de procesamiento de los datos y conservar los resultados generados, de modo que puedan ser recuperados y analizados posteriormente.

2.7.2 Integración entre frontend, backend y agentes

La integración entre el frontend, el backend y los agentes inteligentes es un aspecto fundamental en aplicaciones basadas en arquitecturas distribuidas, ya que permite coordinar el intercambio de información entre la interfaz de usuario y los componentes encargados del procesamiento de datos. Generalmente, el frontend actúa como punto de interacción con el usuario, mientras que el backend implementa la lógica de negocio, expone servicios mediante APIs REST y gestiona la comunicación con otros módulos especializados [45], [46].

En sistemas multi-agente basados en modelos de lenguaje, el backend puede desempeñar el papel de intermediario entre la aplicación web y los agentes inteligentes, recibiendo solicitudes,

orquestando la ejecución de tareas y consolidando los resultados obtenidos. Esta estrategia facilita la incorporación de agentes especializados para actividades como transcripción de audio, análisis de información, recuperación de contexto y generación de documentos, manteniendo desacoplados los componentes de presentación y procesamiento [46], [9].

En este estudio, la plataforma web ofrece al usuario la opción de subir grabaciones de entrevistas de elicitación y acceder a los documentos SRS generados de forma automatizada. Tras recibir el archivo de audio, el backend lo remite al Agente Orquestador, quien se encarga de dirigir la ejecución ordenada de los agentes especializados en transcripción ASR, análisis y categorización, enriquecimiento contextual y redacción del documento SRS. Por último, el informe resultante se guarda en la base de datos y se muestra al usuario en la interfaz web, permitiendo su revisión y comparación en etapas posteriores.

2.7.3 Visualización de resultados y reportes

La visualización de resultados constituye un componente esencial en aplicaciones web modernas, ya que permite presentar información procesada de forma clara, organizada y comprensible para los usuarios. Una adecuada representación de los datos facilita la interpretación de los resultados obtenidos, mejora la experiencia de uso del sistema y contribuye a la toma de decisiones informadas. En aplicaciones de apoyo a la Ingeniería de Software, la presentación estructurada de documentos, métricas y reportes favorece además las actividades de revisión, validación y comparación de artefactos generados automáticamente [45], [46].

Las aplicaciones web actuales suelen incorporar mecanismos de visualización dinámica que permiten mostrar documentos, tablas, indicadores y reportes generados por procesos automatizados. En sistemas basados en inteligencia artificial, estas interfaces posibilitan que los usuarios inspeccionen los resultados producidos por modelos de aprendizaje automático, realicen validaciones manuales y almacenen información para consultas posteriores. Asimismo, la persistencia de los resultados facilita mantener un historial de ejecuciones y efectuar análisis comparativos entre diferentes versiones o documentos generados [45], [5].

En el marco de esta investigación, la interfaz web exhibe el documento SRS producido automáticamente bajo los lineamientos del estándar IEEE 830-1998, ofreciendo al usuario una presentación ordenada de los requisitos funcionales extraídos durante la entrevista de elicitación. Asimismo, el sistema resguarda los informes generados en una base de datos, lo que facilita su recuperación, exportación y confrontación futura con especificaciones realizadas de forma manual, contribuyendo así al proceso de validación del prototipo desarrollado.

CAPÍTULO III

MATERIALES Y MÉTODOS

3.1 Descripción general

El presente capítulo describe el desarrollo del prototipo web para la generación automática de requisitos funcionales de software a partir de audios de entrevistas a Product Owners. El sistema integra cuatro agentes especializados en la transcripción del audio, la extracción y clasificación de requisitos funcionales, la recuperación de contexto académico y la generación de un documento de especificación de requisitos de software (SRS) con una estructura basada en el estándar IEEE 830-1998.

El procesamiento se coordina mediante un orquestador que ejecuta las etapas de forma secuencial. La integración mediante Model Context Protocol (MCP) permite acceder a herramientas de consulta académica, persistencia de información y generación de contenido para el documento. La interfaz web permite gestionar proyectos y entrevistas, consultar los requisitos identificados y su evidencia de origen, visualizar el contexto académico del proyecto y descargar el SRS generado.

3.2 Arquitectura del sistema

La arquitectura del prototipo se organiza en componentes con responsabilidades diferenciadas para transformar el audio de una entrevista de elicitación en un documento de especificación de requisitos de software. Su diseño integra una interfaz web, un backend, un orquestador, cuatro agentes especializados y servicios de almacenamiento y consulta de información.

Esta organización permite ejecutar las etapas del procesamiento de manera individual o mediante un flujo completo. Asimismo, facilita la conservación de los resultados intermedios, como las transcripciones y los requisitos funcionales, para su consulta desde la aplicación.

3.2.1 Arquitectura general del sistema multiagente

El punto de entrada al sistema es una aplicación web desarrollada con Flask. La interfaz se construye mediante plantillas Jinja2, HTML, Bootstrap y CSS, y permite gestionar proyectos, cargar entrevistas y consultar los resultados del procesamiento. Las solicitudes del usuario son atendidas por las rutas del backend, que invocan al agente correspondiente o al orquestador cuando se solicita la ejecución completa.

El orquestador implementa una secuencia de procesamiento definida en Python. Su función consiste en coordinar la ejecución de los agentes de transcripción, análisis de requisitos, enriquecimiento contextual y generación del documento SRS. La coordinación se realiza mediante llamadas a los métodos de estos componentes, sin utilizar un framework externo de orquestación.

Los agentes emplean mecanismos distintos según su responsabilidad. El agente de transcripción utiliza el modelo Whisper para convertir el audio en texto. Los agentes de análisis y enriquecimiento recurren a un modelo de lenguaje local mediante Ollama. Por su parte, la generación del SRS combina contenido producido mediante el modelo de lenguaje con una estructura documental programada y plantillas complementarias.

La integración con las herramientas de consulta académica, acceso a datos y generación de contenido se realiza mediante un cliente MCP y tres servidores especializados. Estos servidores se ejecutan como procesos independientes y se comunican con el cliente mediante el transporte de entrada y salida estándar, denominado `stdio`.

La persistencia se gestiona mediante SQLite. La aplicación utiliza SQLAlchemy para acceder a los datos desde sus componentes web y determinados agentes, mientras que el servidor

MCP de base de datos ejecuta operaciones sobre el mismo archivo de almacenamiento. Los archivos de audio se conservan en el sistema de archivos y su referencia se registra en la base de datos.

3.2.2 Componentes y responsabilidades

La distribución de responsabilidades permite identificar la participación de cada componente dentro del prototipo. La Tabla 1 resume sus funciones principales.

Tabla 1.

Componentes principales del prototipo

Componente	Responsabilidad
Interfaz web	Permitir la gestión de proyectos, la carga de audios y la consulta de transcripciones, requisitos, contexto académico y documentos SRS.
Backend Flask	Atender las solicitudes de la interfaz, validar las entradas y conectar las operaciones del usuario con los componentes de procesamiento.
Orquestador	Ejecutar secuencialmente los cuatro agentes durante el procesamiento completo de una entrevista.
Agente de transcripción	Convertir el audio de la entrevista en texto mediante Whisper y almacenar la transcripción.
Agente de análisis	Identificar requisitos funcionales, registrar actores y prioridades, y analizar su ambigüedad, verificabilidad y respaldo en la entrevista.
Agente de enriquecimiento	Recuperar información académica y generar un contexto general del proyecto respaldado por las fuentes consultadas.
Agente generador SRS	Organizar los requisitos y el contexto del proyecto en un documento con estructura basada en IEEE 830-1998.
Cliente y servidores MCP	Proporcionar acceso estructurado a herramientas de consulta académica, persistencia y generación de contenido documental.
Almacenamiento	Conservar los datos de los proyectos y los resultados del procesamiento en SQLite, junto con los audios almacenados como archivos.

Fuente: Elaboración propia.

Los componentes comparten información mediante los datos almacenados. Las transcripciones y los requisitos mantienen su relación con la entrevista de origen, mientras que el contexto académico y los documentos SRS se asocian al proyecto.

3.2.3 Flujo de procesamiento de la información

El procesamiento inicia con la creación de un proyecto y la carga de una entrevista en formato de audio. A partir de esta entrada, el sistema desarrolla las siguientes etapas:

1. **Registro de la entrevista.** El backend verifica que el archivo corresponda a uno de los formatos admitidos, lo almacena con un nombre único y registra su asociación con el proyecto.
2. **Transcripción del audio.** El agente de transcripción procesa el archivo mediante Whisper y almacena el texto obtenido. Esta transcripción constituye la entrada para el análisis de requisitos.
3. **Extracción y clasificación de requisitos.** El agente analista procesa la transcripción para identificar requisitos funcionales. Cada requisito se registra con su descripción, actor, prioridad, indicadores de ambigüedad y verificabilidad, y un fragmento de evidencia de la entrevista.
4. **Recuperación del contexto académico.** El agente de enriquecimiento consulta publicaciones mediante Crossref y actualiza el contexto académico del proyecto a partir de la evidencia disponible.
5. **Generación del documento SRS.** El agente generador organiza los requisitos registrados y el contexto disponible en una especificación con estructura basada en IEEE 830-1998.
6. **Almacenamiento y presentación.** El documento generado se almacena en la base de datos y se presenta en la interfaz web. Adicionalmente, el usuario puede descargarlo en formato Word.

Cuando se solicita el procesamiento completo, el orquestador ejecuta estas etapas en el orden establecido. La interfaz también permite ejecutar etapas de manera independiente. La actualización del contexto académico considera los requisitos disponibles del proyecto, mientras que la generación de un nuevo SRS produce un registro documental adicional.

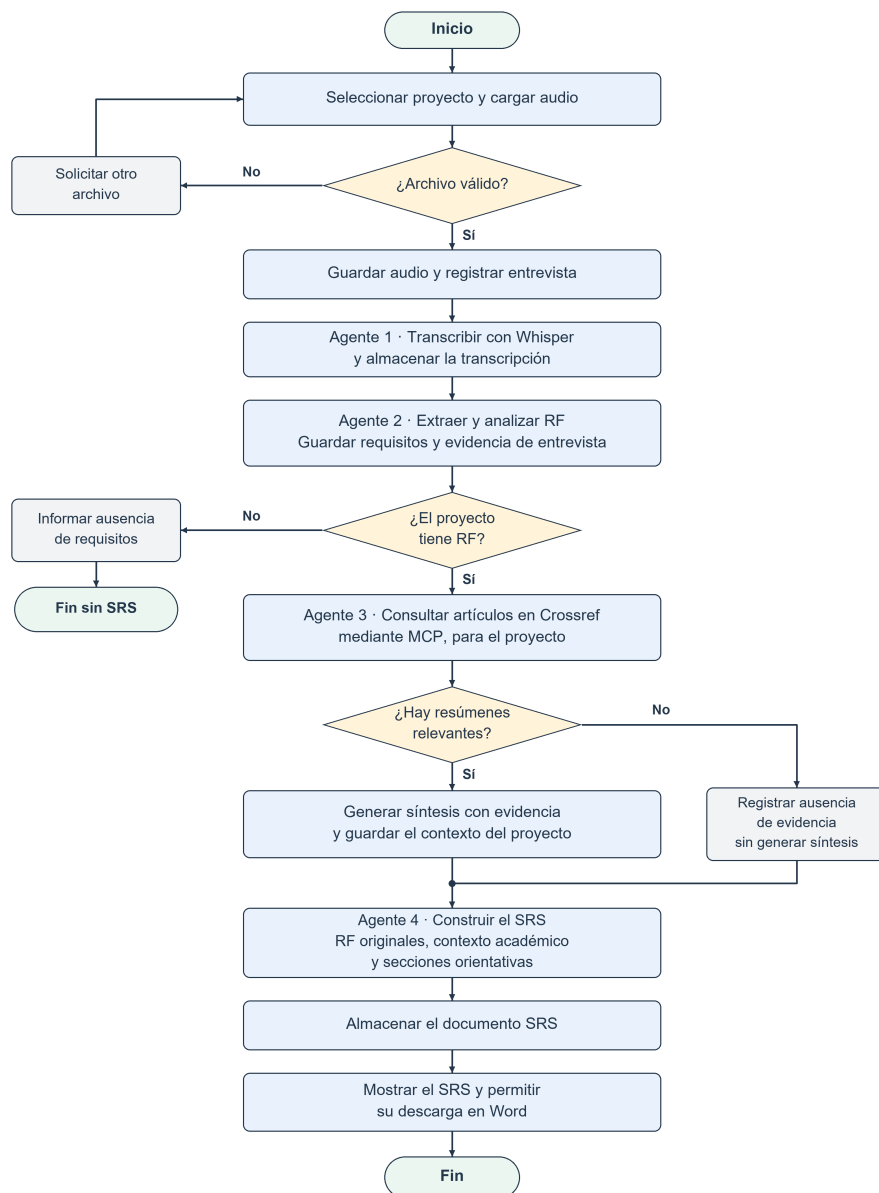
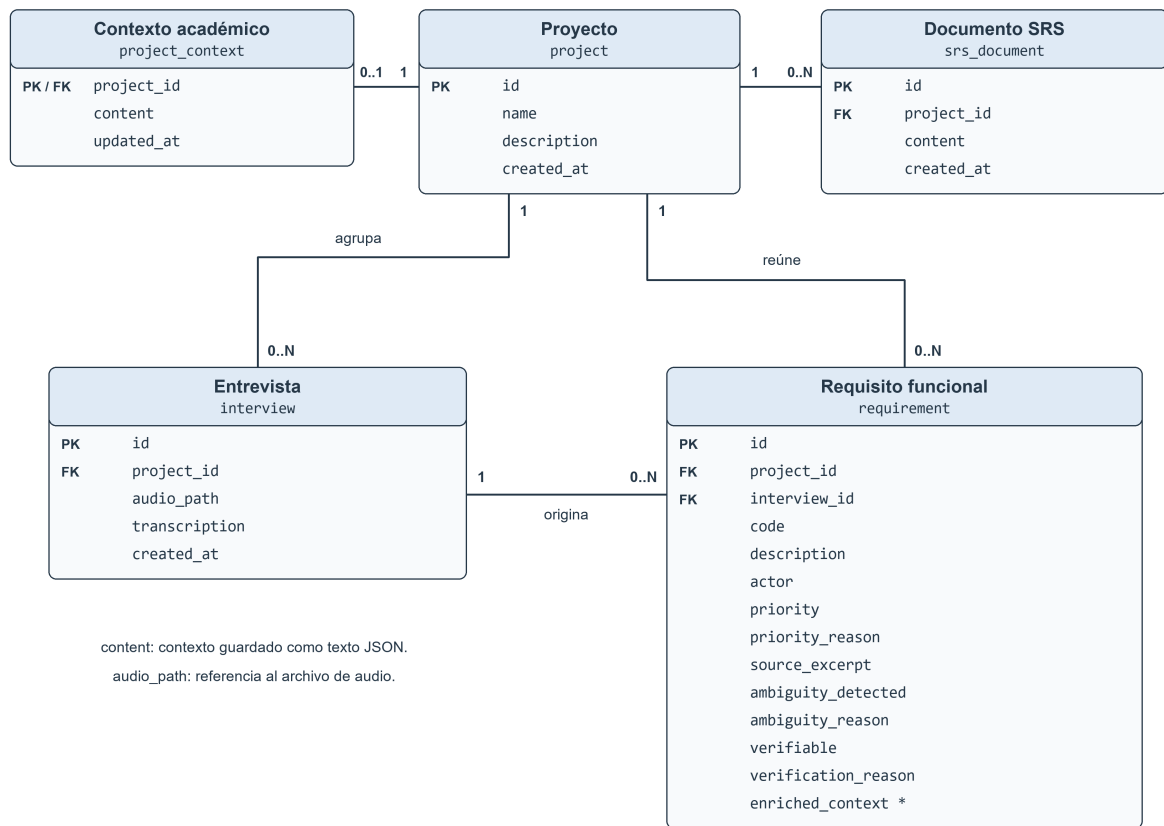


Figura 7.
Flujo de procesamiento para la generación del documento SRS.

3.2.4 Diseño de la base de datos

El prototipo utiliza SQLite para almacenar la información mediante cinco entidades principales: proyecto, entrevista, requisito funcional, contexto académico y documento SRS. El proyecto constituye el punto de asociación de los datos y resultados del procesamiento.



PK: clave primaria · FK: clave foránea · 0..N: cero o varios registros · 0..1: cero o un registro

Figura 8.

Diseño de la base de datos

Cada proyecto puede contener varias entrevistas y cada entrevista puede originar múltiples requisitos funcionales. Los requisitos conservan sus atributos y el fragmento de transcripción que respalda su identificación, manteniendo la trazabilidad hacia la entrevista de origen.

El contexto académico se almacena como texto en formato JSON en un registro único por proyecto, que se actualiza al ejecutar nuevamente el enriquecimiento.

Los archivos de audio se conservan en el sistema de archivos; la base de datos almacena la referencia que permite localizarlos.

3.3 Implementación del protocolo MCP

El protocolo Model Context Protocol (MCP) se incorporó al prototipo para proporcionar acceso estructurado a herramientas de consulta académica, persistencia de información y generación de contenido documental. La integración comprende un cliente común y tres servidores especializados.

La implementación utiliza el SDK de MCP para Python, mediante el paquete `mcp` en su versión 2.1.1, verificada en el entorno del proyecto. Esta numeración corresponde a la versión del paquete instalado, no a la versión del protocolo.

3.3.1 Organización de la integración MCP

La integración se organiza mediante la clase `MCPClient`, ubicada en `app/mcp/client.py`. Esta clase concentra las llamadas a las herramientas de los servidores de consulta académica, base de datos y generación SRS.

El agente de enriquecimiento utiliza estas herramientas para consultar la información del proyecto, recuperar publicaciones académicas y guardar el contexto generado. El agente generador las utiliza para obtener los datos necesarios, solicitar contenido general y almacenar el documento SRS.

3.3.2 Cliente MCP y comunicación con los servidores

El cliente centraliza la comunicación mediante el método `_call_tool`, que recibe la ruta del servidor, el nombre de la herramienta y sus argumentos. El proceso se configura mediante `StdioServerParameters`, indicando el intérprete de Python y el archivo del servidor que se ejecutará.

La comunicación utiliza el transporte `stdio`, basado en la entrada y salida estándar de los

procesos. El cliente abre un contexto asíncrono mediante `Client`, invoca la herramienta con `call_tool` y recupera su respuesta.

Cada operación comprende la selección del servidor, el establecimiento de la comunicación, la invocación de la herramienta y la comprobación del resultado. Cada llamada abre su propio contexto de comunicación, que se cierra al finalizar la operación.

Los métodos síncronos del prototipo ejecutan estas operaciones asíncronas mediante `asyncio.run`. De esta manera, los agentes utilizan las herramientas sin gestionar individualmente las conexiones MCP.

La Figura 9 representa el intercambio entre el componente solicitante, el cliente MCP, el servidor y el servicio utilizado para atender una operación.

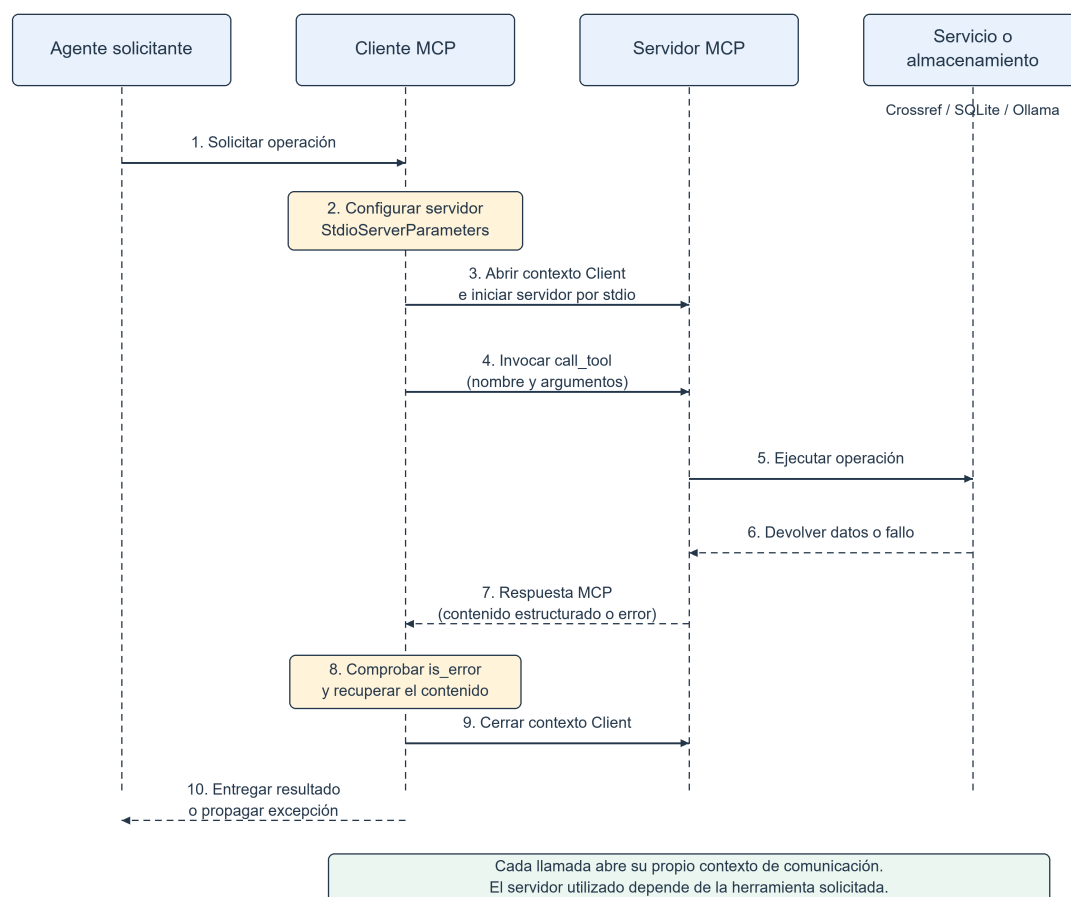


Figura 9.
Secuencia de comunicación para la ejecución de herramientas MCP.

Fuente: Elaboración propia.

3.3.3 Servidor MCP de consulta académica

El servidor de consulta académica, implementado en `web_server.py`, expone la herramienta `search_academic`. Esta recibe una consulta textual y un límite de resultados, y utiliza `AcademicSearchService` para acceder a Crossref.

La respuesta se organiza mediante el modelo `AcademicSearchResponse`, que incluye los registros recuperados, sus metadatos, los resúmenes disponibles y las advertencias de la consulta. La selección de las publicaciones y la síntesis del contexto corresponden al procesamiento posterior del agente de enriquecimiento.

3.3.4 Servidor MCP de base de datos

El servidor `database_server.py` utiliza el módulo `sqlite3` para ejecutar consultas parametrizadas sobre la base de datos del prototipo. Sus herramientas permiten recuperar entrevistas, requisitos e información del proyecto, así como almacenar el contexto académico y los documentos SRS.

El contexto se actualiza en un registro único por proyecto, mientras que cada documento SRS generado se almacena como un registro adicional. Las operaciones de escritura confirman los cambios y las conexiones se cierran al finalizar su ejecución.

3.3.5 Servidor MCP de generación SRS

El servidor `srs_server.py` expone la herramienta `generar_informacion_srs`, que recibe información del proyecto y sus requisitos funcionales. Mediante el servicio de Ollama, consulta el modelo `llama3.2:3b` y devuelve contenido general organizado según un esquema JSON.

Esta herramienta apoya la redacción de información descriptiva. La construcción del docu-

mento final, la incorporación de los requisitos registrados y la organización de sus apartados corresponden al agente generador.

La Tabla 2 resume las principales herramientas utilizadas por el prototipo y la información que intercambian.

Tabla 2.

Principales herramientas MCP

Servidor	Herramienta	Entrada y salida
Consulta académica	<code>search_academic</code>	Entrada: consulta textual y límite de resultados. Salida: registros académicos, metadatos, resúmenes disponibles y advertencias de la consulta.
Base de datos	<code>get_interview</code>	Entrada: identificador de entrevista. Salida: datos de la entrevista, incluida su transcripción.
Base de datos	<code>get_requirements</code>	Entrada: identificador de entrevista. Salida: requisitos funcionales registrados para esa entrevista.
Base de datos	<code>get_project_srs_data</code>	Entrada: identificador de proyecto. Salida: información del proyecto, sus entrevistas, requisitos funcionales y contexto académico.
Base de datos	<code>save_project_context</code>	Entrada: identificador de proyecto y contexto académico en formato JSON. Salida: confirmación del almacenamiento o actualización del contexto.
Base de datos	<code>save_srs_document</code>	Entrada: identificador de proyecto y contenido del documento SRS. Salida: identificador del documento almacenado.
Generación SRS	<code>generar_informacion_srs</code>	Entrada: nombre, descripción y requisitos funcionales del proyecto. Salida: contenido general estructurado para apoyar la elaboración del documento SRS.

Fuente: Elaboración propia.

3.4 Implementación de los agentes especializados y del orquestador

El prototipo distribuye el procesamiento en cuatro agentes especializados: transcripción, extracción de requisitos funcionales, enriquecimiento académico y generación del documento SRS. Cada agente consulta la información necesaria, realiza su tarea y conserva los resultados para las etapas posteriores. El orquestador coordina su ejecución cuando el usuario solicita el procesamiento completo de una entrevista.

La Tabla 3 relaciona cada componente con sus principales herramientas y modelos. Las versiones corresponden al entorno verificado del proyecto; los identificadores de los modelos se distinguen de las versiones de las bibliotecas y los servicios que los ejecutan.

Tabla 3.

Herramientas y modelos utilizados por los agentes y el orquestador

Componente	Herramientas principales	Versión o modelo
Agente de transcripción	Whisper para convertir audio en texto, con soporte de PyTorch.	openai-whisper: 20250625. Modelo: base. PyTorch: 2.14.0.
Agente de extracción de requisitos	Ollama para ejecutar el modelo de lenguaje y reglas de validación implementadas en Python.	Ollama: 0.34.2. Modelo: llama3.2:3b.
Agente de enriquecimiento académico	Ollama para la síntesis; herramientas MCP para consultar Crossref y acceder a los datos del proyecto.	Ollama: 0.34.2. Modelo: llama3.2:3b. SDK MCP: 2.1.1.
Agente generador SRS	Herramientas MCP de persistencia y generación documental; Ollama mediante el servidor SRS y plantillas programadas.	SDK MCP: 2.1.1. Ollama: 0.34.2. Modelo: llama3.2:3b.
Orquestador	Coordinación secuencial mediante llamadas a los agentes, sin un framework externo de orquestación.	Python: 3.14.7.

Fuente: Elaboración propia a partir del código y del entorno creado en el proyecto. La versión del SDK corresponde al paquete mcp. Crossref se utiliza como servicio externo, sin una versión instalada localmente.

3.4.1 Agente de transcripción

El agente de transcripción convierte el audio de la entrevista en texto mediante Whisper, utilizando el modelo base ejecutado localmente. Antes de iniciar, comprueba que la entrevista se encuentre registrada, que disponga de un archivo de audio asociado y que dicho archivo exista

en el almacenamiento del sistema.

La transcripción se configura en idioma español. Una vez finalizado el procesamiento, el agente comprueba que el texto obtenido no esté vacío y lo almacena en el registro de la entrevista. Este resultado constituye la entrada del agente encargado de identificar los requisitos funcionales.

Si no se encuentra el archivo o no se obtiene una transcripción válida, el agente comunica el error e impide que el procesamiento continúe con una entrada incompleta.

3.4.2 Agente de extracción y clasificación de requisitos funcionales

Este agente analiza la transcripción para identificar funcionalidades solicitadas durante la entrevista. Para ello, divide el texto en segmentos y aplica un filtro basado en verbos de acción y expresiones que pueden representar necesidades del usuario. Los segmentos seleccionados se procesan mediante el modelo llama3.2:3b, ejecutado localmente a través de Ollama.

Las instrucciones proporcionadas al modelo delimitan la extracción a la información presente en cada segmento. Se solicita evitar la incorporación de funcionalidades, actores u objetos que no estén expresados en la entrevista, así como excluir enunciados que correspondan únicamente a requisitos no funcionales. La respuesta se solicita en formato JSON para facilitar su interpretación y validación mediante código.

Los candidatos obtenidos se someten a controles de respaldo en el texto de origen y de similitud con los requisitos previamente aceptados, con el propósito de reducir incorporaciones sin evidencia y duplicaciones. Además, se registra el actor cuando puede identificarse en la entrevista y se asigna la prioridad a partir de las expresiones presentes en el texto.

El agente también aplica reglas para señalar posibles ambigüedades y dificultades de verificación. Estos indicadores apoyan la revisión posterior de los requisitos; no constituyen una garantía de su corrección. Cada requisito aceptado se almacena junto con el fragmento de la en-

trevista que respalda su identificación, manteniendo su asociación con la entrevista y el proyecto correspondiente.

3.4.3 Agente de enriquecimiento académico por proyecto

El agente de enriquecimiento genera un contexto académico compartido para el proyecto. Para construir las consultas, considera el nombre y la descripción del proyecto, las transcripciones disponibles y los requisitos funcionales registrados. A partir de esta información, identifica términos del dominio y de las funcionalidades solicitadas.

La recuperación automática se realiza mediante Crossref, a través de la herramienta MCP de consulta académica. Los resultados se filtran y ordenan utilizando la coincidencia de términos relacionados con el dominio, las funcionalidades y el software. También se eliminan duplicados mediante el DOI o la dirección de la publicación. El sistema proporciona adicionalmente un enlace de consulta en Google Scholar para apoyar la búsqueda manual.

La síntesis utiliza las publicaciones seleccionadas que disponen de resumen. Mediante Ollama, el modelo llama3.2:3b recibe instrucciones para proponer un máximo de cinco aportes contextuales, indicando la fuente y el fragmento de evidencia que los respalda. Estos aportes deben complementar la comprensión del proyecto sin introducir nuevos requisitos funcionales.

El resultado se almacena como un único contexto asociado al proyecto, junto con sus fuentes y las advertencias de la búsqueda. Cuando no se recupera evidencia suficiente, el sistema registra esta situación y permite continuar con la generación documental sin una síntesis académica respaldada.

3.4.4 Agente generador del documento SRS

El agente generador consulta los datos del proyecto, los requisitos funcionales y el contexto académico almacenado para construir una especificación con estructura basada en IEEE

830-1998. La elaboración combina contenido generado mediante el modelo de lenguaje con secciones organizadas mediante código y plantillas.

La solicitud de contenido general se realiza mediante el servidor MCP de generación SRS, que utiliza Ollama para consultar el modelo llama3.2:3b. El agente incorpora la respuesta a la estructura documental y reúne la información registrada del proyecto.

Los requisitos funcionales se incorporan a partir de los registros existentes, conservando sus descripciones y los atributos disponibles. El documento incluye también los fragmentos de entrevista que respaldan su identificación y el contexto académico del proyecto, cuando este se encuentra disponible.

Debido a que la tesis se concentra en los requisitos funcionales, los demás apartados se completan con contenido orientativo relacionado con el proyecto. Estas secciones incluyen la indicación (*Fuera del alcance de la tesis*), para distinguirlas de los requisitos obtenidos de las entrevistas. Su contenido sirve como guía de elaboración y requiere revisión antes de utilizarse como una especificación definitiva.

Finalmente, el agente almacena el documento generado mediante las herramientas MCP de persistencia y devuelve su identificador. La exportación a Word se realiza posteriormente mediante un servicio complementario que utiliza python-docx, versión 1.2.0.

3.4.5 Orquestador del sistema

El orquestador coordina el procesamiento completo a partir del identificador de una entrevista. Primero comprueba su existencia y obtiene el proyecto al que pertenece. Posteriormente, ejecuta los agentes en el siguiente orden:

1. Transcripción del audio de la entrevista.
2. Extracción y almacenamiento de los requisitos funcionales.

3. Actualización del contexto académico utilizando la información acumulada del proyecto.
4. Generación y almacenamiento del documento SRS.

La continuidad entre etapas se apoya en los identificadores y en los resultados almacenados en la base de datos. De esta manera, la transcripción y la extracción operan sobre la entrevista seleccionada, mientras que el enriquecimiento y la generación documental consideran el conjunto de requisitos del proyecto.

Cuando una etapa produce un error que no se resuelve internamente, se interrumpe la ejecución de las etapas posteriores. Los resultados que ya fueron guardados permanecen disponibles. Al completar el proceso, el orquestador devuelve los identificadores del proyecto, de la entrevista y del documento generado, junto con la cantidad de requisitos obtenidos.

La Figura 10 organiza las actividades por responsable y muestra las principales decisiones del procesamiento. Los carriles distinguen las operaciones realizadas por entrevista de aquellas que consideran el proyecto completo.

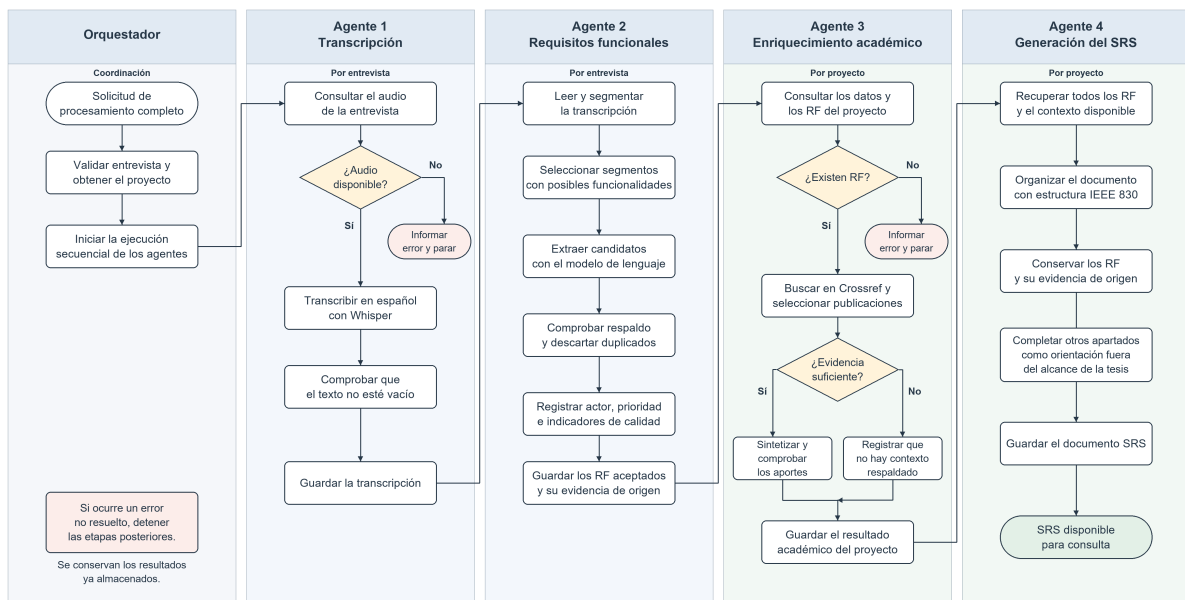


Figura 10.

Diagrama de actividades del procesamiento multiagente.

Fuente: Elaboración propia.

3.5 Tecnologías y entorno de ejecución

Esta sección describe las características del equipo, las herramientas de software y la configuración utilizada para ejecutar el prototipo. Su documentación permite identificar las condiciones técnicas del entorno de desarrollo y facilita la reproducción de su configuración.

3.5.1 Características del equipo utilizado

El desarrollo y la ejecución local del prototipo se realizaron en un equipo cuyas características se presentan en la Tabla 4. Estas especificaciones corresponden al equipo utilizado y no representan los requisitos mínimos de funcionamiento del sistema.

Tabla 4.

Características del equipo utilizado

Componente	Característica
Procesador	AMD Ryzen 5 5600X de 6 núcleos.
Memoria RAM	24 GB instalados.
Tarjeta gráfica	AMD Radeon R9 Fury Series, con aproximadamente 4 GB de memoria de video.
Almacenamiento del proyecto	Unidad SSD HS-SSD-WAVE(P) de 512 GB nominales, correspondiente al disco donde se encuentra el proyecto.

Fuente: Elaboración propia a partir de las características consultadas en el equipo.

La presencia de una tarjeta gráfica se registra como característica del hardware disponible. Por sí sola, esta información no demuestra que los modelos hayan utilizado aceleración gráfica durante el procesamiento.

3.5.2 Entorno de software y herramientas utilizadas

El prototipo se implementó en Python y utiliza un entorno virtual para gestionar sus dependencias. Flask permite atender las solicitudes de la aplicación web, mientras que las plantillas Jinja2 y los estilos de Bootstrap participan en la presentación de la interfaz.

La Tabla 5 recoge las versiones verificadas en el entorno del proyecto. En el caso de los modelos de inteligencia artificial, se presenta su identificador por separado de la versión de la herramienta que los ejecuta.

Tabla 5.

Software y modelos utilizados en el prototipo

Componente	Versión o modelo	Función
Sistema operativo	Windows 11 Pro, 64 bits; compilación 26200	Entorno de ejecución del prototipo.
Python	3.14.7	Lenguaje de implementación.
Flask	3.1.3	Backend y rutas de la aplicación web.
Jinja2	3.1.6	Renderización de plantillas HTML.
Bootstrap	5.3.3	Estilos y distribución visual de la interfaz.
Flask-SQLAlchemy	3.1.1	Integración de la persistencia con Flask.
SQLAlchemy	2.0.52	Mapeo y acceso a los datos.
SQLite	3.50.4	Motor de base de datos local.
SDK de MCP para Python	2.1.1	Implementación del cliente y los servidores MCP.
Ollama	0.34.2	Ejecución local del modelo de lenguaje.
Modelo de lenguaje	llama3.2:3b	Extracción de requisitos y generación de texto.
OpenAI Whisper	20250625; modelo base	Transcripción local de audio.
PyTorch	2.14.0	Soporte de ejecución para Whisper.
python-docx	1.2.0	Construcción del archivo Word exportable.

Fuente: Elaboración propia a partir de las versiones instaladas y la configuración del proyecto.

La interfaz se complementa con HTML y CSS propio. Por otra parte, Crossref se utiliza como servicio externo para la recuperación de información académica, por lo que no corresponde a una dependencia instalada con una versión local dentro del proyecto.

3.5.3 Configuración de ejecución

El prototipo se ejecuta de manera local en el equipo descrito anteriormente. La aplicación web permite acceder a las funciones del sistema, mientras que la información de los proyectos, las entrevistas y los requisitos se conserva en una base de datos SQLite. Los archivos de audio se almacenan en una carpeta del proyecto.

Para el procesamiento de texto se utiliza Ollama con el modelo llama3.2:3b. Durante la ejecución del prototipo, el servicio se inicia desde PowerShell con una configuración orientada al uso del procesador del equipo (CPU). De esta manera, el procesamiento del modelo de lenguaje se realiza localmente.

La transcripción de las entrevistas se realiza mediante Whisper con el modelo base, configurado para reconocer audio en español. Este componente también se ejecuta de forma local y transforma las grabaciones en texto para su posterior análisis.

Aunque los modelos se ejecutan en el equipo, se requiere conexión a Internet para consultar información académica mediante Crossref y cargar los estilos de Bootstrap referenciados por la interfaz web.

Esta configuración describe las condiciones de ejecución del prototipo. Los tiempos de procesamiento y los resultados de su evaluación se presentan en el capítulo de resultados y análisis.

3.6 Metodología de desarrollo: adaptación de Scrum al proyecto

Para describir la organización del desarrollo del prototipo, se plantea una adaptación de Scrum al contexto de un trabajo de titulación individual. Esta adaptación organiza las funcionalidades mediante un Product Backlog y distribuye su implementación en incrementos que permiten integrar progresivamente los componentes del sistema.

La organización metodológica considera las características del proyecto: un único desarrollador, la orientación académica del director de tesis y la construcción de un prototipo centrado en la generación de requisitos funcionales. La distribución de las actividades y los tiempos de los cinco sprints se detalla en la siguiente sección.

3.6.1 Adaptación de Scrum al desarrollo individual

El desarrollo individual concentra las actividades de análisis, diseño, programación e integración en el autor del proyecto. Por esta razón, la adaptación se enfoca en la priorización del trabajo, la definición de entregables y la revisión de los incrementos, sin establecer una separación artificial de responsabilidades entre varios integrantes.

Cada incremento se define a partir de una capacidad comprobable del prototipo. Esto permite organizar el avance desde las funciones iniciales de gestión y almacenamiento hasta la integración de los agentes y la generación del documento SRS.

Las dependencias entre componentes condicionan el orden del trabajo. Por ejemplo, la extracción de requisitos requiere una transcripción disponible, mientras que el enriquecimiento académico y la generación documental necesitan los requisitos registrados en el proyecto.

3.6.2 Responsabilidades de los participantes

La Tabla 6 presenta a los participantes del trabajo de titulación y sus responsabilidades, diferenciando el desarrollo técnico, la dirección académica.

Tabla 6.

Participantes y responsabilidades en el proyecto

Participante	Función	Responsabilidades
Andrés Sebastian Haro Osnayo	Autor y desarrollador del prototipo	Analizar las funcionalidades, diseñar e implementar la solución, integrar los componentes, comprobar su funcionamiento y documentar el trabajo.
Dra. Cathy Pamela Guevara Vega	Directora del trabajo de titulación	Orientar el desarrollo académico y metodológico, revisar el informe y verificar su correspondencia con los objetivos de la investigación.
MSc. Pablo Andrés Landeta	Acesor del trabajo de titulación	Participar en la revisión y calificación del trabajo de titulación como acesor.

Fuente: Elaboración propia a partir de la identificación de los participantes del trabajo de titulación.

3.6.3 Definición del Product Backlog

El Product Backlog reúne las funcionalidades y tareas técnicas necesarias para construir el prototipo. Sus elementos corresponden al desarrollo del sistema y se distinguen de los requisitos funcionales que este extrae de las entrevistas.

La Tabla 7 presenta una organización del trabajo en cinco sprints. La asignación indica el sprint principal de cada elemento, aunque su integración puede requerir ajustes en etapas posteriores.

La prioridad alta corresponde a los elementos necesarios para el procesamiento principal y

la integración del sistema. La prioridad media identifica funcionalidades complementarias para utilizar los resultados. Los criterios de aceptación se detallan en la descripción de cada sprint.

Tabla 7.

Product Backlog del prototipo

ID	Elemento del backlog	Sprint	Prioridad
PB-01	Configurar el entorno Python, las dependencias y la estructura inicial de la aplicación Flask.	1	Alta
PB-02	Definir los modelos de datos y sus relaciones mediante SQLAlchemy y SQLite.	1	Alta
PB-03	Implementar el registro de proyectos y entrevistas, con carga, validación y almacenamiento de audios.	1	Alta
PB-04	Integrar Whisper e implementar el agente de transcripción, conservando el texto obtenido.	1	Alta
PB-05	Integrar Ollama y definir las instrucciones para extraer requisitos funcionales en formato JSON.	2	Alta
PB-06	Implementar la validación y el almacenamiento de requisitos, incluyendo actores, prioridades, evidencia e indicadores de calidad.	2	Alta
PB-07	Construir el orquestador inicial para conectar la transcripción con la extracción de requisitos.	2	Alta
PB-08	Implementar el cliente MCP y el servidor de consulta académica mediante Crossref.	3	Alta
PB-09	Construir consultas relacionadas con el proyecto y seleccionar publicaciones pertinentes.	3	Alta
PB-10	Implementar el agente de enriquecimiento para generar un contexto académico por proyecto, con fuentes y comprobaciones de evidencia.	3	Alta

Continúa en la siguiente página.

Cuadro 7: Product Backlog del prototipo (continuación)

ID	Elemento del backlog	Sprint	Prioridad
PB-11	Implementar el servidor MCP de base de datos para consultar y guardar la información requerida por los agentes.	3	Alta
PB-12	Implementar el servidor MCP de generación de contenido documental y el agente SRS.	4	Alta
PB-13	Organizar el documento según IEEE 830-1998, conservando los RF e identificando los apartados orientativos fuera del alcance.	4	Alta
PB-14	Integrar el contexto académico y la evidencia de las entrevistas en el SRS, y almacenar el documento generado.	4	Alta
PB-15	Implementar la exportación del documento SRS a formato Word.	4	Media
PB-16	Completar la interfaz web para gestionar proyectos y entrevistas y consultar los resultados.	5	Alta
PB-17	Integrar los cuatro agentes con el orquestador y la interfaz para ejecutar el proceso completo.	5	Alta
PB-18	Comprobar el funcionamiento de extremo a extremo, el manejo de errores y la conservación de los resultados, y corregir los problemas detectados.	5	Alta

Fuente: Elaboración propia. Distribución propuesta a partir de las funcionalidades implementadas.

La distribución considera las dependencias del prototipo. La persistencia básica se incorpora desde el primer sprint, mientras que el servidor MCP de base de datos se ubica junto al enriquecimiento académico, debido a que este componente permite consultar la información del proyecto y almacenar su contexto.

Los primeros sprints concentran el procesamiento de entrevistas y requisitos; los siguientes incorporan el enriquecimiento y la generación documental. El último sprint reúne la interfaz, la integración completa y las comprobaciones de funcionamiento previas a la evaluación del sistema.

3.6.4 Seguimiento y revisión de los incrementos

Para documentar el seguimiento del desarrollo, cada incremento se relaciona con las funcionalidades del backlog que incorpora y con sus criterios de aceptación. La revisión considera el comportamiento observable del sistema: las entradas que recibe, los resultados que produce y la información que conserva.

Un incremento se considera completado cuando la funcionalidad prevista está implementada, se integra con los componentes necesarios y satisface el criterio de aceptación definido. Cuando una revisión identifica un problema, se registra el ajuste pendiente y su relación con la funcionalidad afectada.

La documentación de cada sprint debe recoger su objetivo, las actividades realizadas, el entregable obtenido y los ajustes necesarios. Esta información permite explicar la evolución del prototipo sin repetir la descripción técnica de los agentes.

Las comprobaciones realizadas durante el desarrollo se orientan a verificar el funcionamiento y la integración de los componentes. La evaluación de la calidad de los requisitos generados y el análisis de los resultados se presentan en el capítulo IV.

3.7 Planificación y desarrollo de los sprints

El periodo de desarrollo considerado comprende desde el 31 de julio hasta el 7 de septiembre de 2026, con una duración de 39 días calendario. Para documentar el trabajo, se establece una distribución retrospectiva en cinco sprints, manteniendo la estructura general de la propuesta

metodológica y adaptando las actividades a la implementación del prototipo.

Los primeros cuatro sprints comprenden ocho días calendario cada uno y el último comprende siete días. Esta distribución organiza los componentes implementados según sus dependencias y no representa un registro diario de su construcción. La Tabla 8 presenta los periodos asignados.

Tabla 8.
Distribución de los sprints

Sprint	Enfoque	Periodo	Duración
1	Configuración y transcripción	31/07/2026 – 07/08/2026	8 días
2	Extracción de requisitos y orquestador inicial	08/08/2026 – 15/08/2026	8 días
3	Integración MCP y enriquecimiento académico	16/08/2026 – 23/08/2026	8 días
4	Generación del documento SRS	24/08/2026 – 31/08/2026	8 días
5	Interfaz, integración y pruebas	01/09/2026 – 07/09/2026	7 días

Fuente: Elaboración propia. Distribución retrospectiva del periodo de desarrollo.

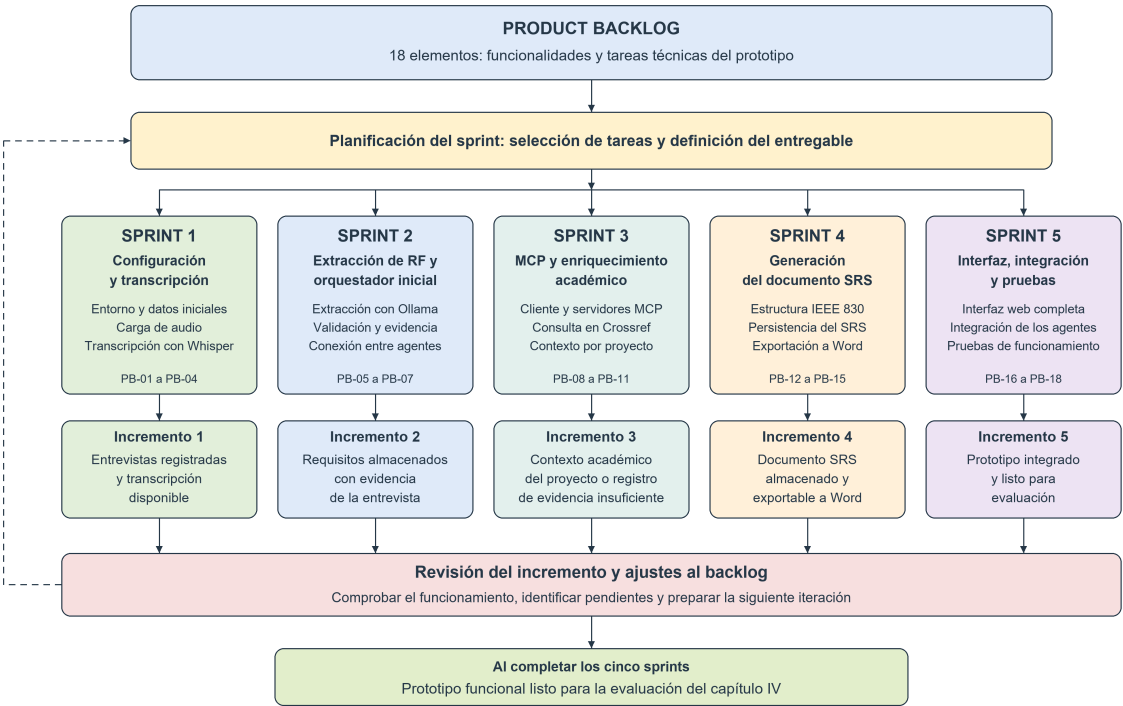


Figura 11.
Organización del desarrollo del prototipo mediante cinco sprints

En los siguientes apartados se describen el objetivo, las tareas y el incremento asociado a

cada sprint. Los criterios de aceptación permiten comprobar los entregables; su inclusión no sustituye la presentación de evidencias de prueba ni la evaluación de calidad del capítulo IV.

3.7.1 Sprint 1: configuración y transcripción

Periodo: 31 de julio al 7 de agosto de 2026 (8 días calendario).

Objetivo: Establecer la base técnica del prototipo y disponer del procesamiento necesario para registrar entrevistas y convertir sus archivos de audio en texto.

Este sprint agrupa la configuración del entorno, la persistencia inicial y la integración de Whisper. El almacenamiento se incorpora desde esta etapa para conservar los proyectos, las entrevistas y sus transcripciones.

Tabla 9.

Tareas y criterios de aceptación del sprint 1

ID	Tarea	Criterio de aceptación
PB-01	Configurar Python, las dependencias y la aplicación Flask.	La aplicación inicia desde el entorno configurado y permite acceder a sus rutas.
PB-02	Definir los modelos y las relaciones mediante SQLAlchemy y SQLite.	Los proyectos y las entrevistas se guardan y recuperan conservando su asociación.
PB-03	Implementar el registro de proyectos, entrevistas y archivos de audio.	Un audio válido se almacena y se vincula con una entrevista del proyecto seleccionado. Las entradas no admitidas se rechazan.
PB-04	Integrar el agente de transcripción con Whisper.	El audio genera una transcripción no vacía, que se almacena en la entrevista. La ausencia de audio se comunica como error.

Fuente: Elaboración propia.

Resultado del sprint: Se obtuvo la base funcional del prototipo, con registro de proyectos y entrevistas, almacenamiento de audios y transcripción mediante Whisper. El texto generado queda asociado a la entrevista y disponible para la extracción de requisitos.

3.7.2 Sprint 2: extracción de requisitos y orquestador inicial

Periodo: 8 al 15 de agosto de 2026 (8 días calendario).

Objetivo: Transformar las transcripciones en requisitos funcionales estructurados y conectar las primeras etapas del procesamiento.

Las tareas de este sprint comprenden la integración del modelo de lenguaje, las instrucciones de extracción y los controles aplicados a los candidatos. La conexión inicial del orquestador permite encadenar la transcripción con el análisis de requisitos.

Tabla 10.

Tareas y criterios de aceptación del sprint 2

ID	Tarea	Criterio de aceptación
PB-05	Integrar Ollama y definir las instrucciones de extracción.	El servicio devuelve candidatos en un JSON interpretable. Una respuesta vacía o con formato inválido se identifica como error.
PB-06	Validar y almacenar los requisitos y sus atributos.	Los requisitos aceptados conservan su relación con la entrevista y su evidencia. Se aplican controles de respaldo, duplicación e indicadores de calidad.
PB-07	Conectar la transcripción y la extracción mediante el orquestador inicial.	La extracción utiliza la transcripción guardada de la entrevista seleccionada y no continúa si falla la etapa anterior.

Fuente: Elaboración propia.

Resultado del sprint: Se incorporó la extracción de requisitos funcionales mediante Ollama y su almacenamiento con los atributos y fragmentos de entrevista que los respaldan. El orquestador inicial conecta la transcripción con el análisis de requisitos.

3.7.3 Sprint 3: integración MCP y enriquecimiento académico

Periodo: 16 al 23 de agosto de 2026 (8 días calendario).

Objetivo: Incorporar la consulta académica mediante MCP y generar un contexto compartido para el proyecto.

Este sprint reúne la comunicación con las herramientas de búsqueda y persistencia, la selección de publicaciones y la síntesis de información respaldada por las fuentes. El enriquecimiento considera los requisitos acumulados del proyecto y registra cuando no existe evidencia suficiente para producir una síntesis.

Tabla 11.

Tareas y criterios de aceptación del sprint 3

ID	Tarea	Criterio de aceptación
PB-08	Implementar el cliente MCP y el servidor de consulta académica.	El cliente invoca la herramienta de búsqueda y recibe resultados estructurados o información sobre los problemas de consulta.
PB-09	Construir consultas y seleccionar publicaciones.	Las consultas emplean información del proyecto y sus RF. Los resultados se filtran por pertinencia y se eliminan duplicados.
PB-10	Implementar el enriquecimiento académico por proyecto.	El contexto conserva fuentes y evidencia. Cuando esta resulta insuficiente, se registra la situación sin incorporar una síntesis sin respaldo.
PB-11	Implementar las herramientas MCP de persistencia.	Los agentes consultan los datos del proyecto y guardan su contexto académico mediante las herramientas correspondientes.

Fuente: Elaboración propia.

Resultado del sprint: Se integraron las herramientas MCP de consulta académica y persistencia con el agente de enriquecimiento. El sistema permite almacenar un contexto general por proyecto, acompañado de sus fuentes, o registrar la ausencia de evidencia suficiente para generarlo.

3.7.4 Sprint 4: generación del documento SRS

Periodo: 24 al 31 de agosto de 2026 (8 días calendario).

Objetivo: Construir una especificación basada en IEEE 830-1998 a partir de los requisitos y del contexto disponible, y permitir su almacenamiento y exportación.

El documento integra los requisitos registrados sin sustituir sus descripciones por nuevas funcionalidades generadas. Los apartados que exceden el alcance de la tesis se completan como orientación y se identifican expresamente.

Tabla 12.

Tareas y criterios de aceptación del sprint 4

ID	Tarea	Criterio de aceptación
PB-12	Implementar el servidor MCP documental y el agente generador SRS.	El agente obtiene la información del proyecto e invoca la herramienta de generación de contenido general.
PB-13	Organizar la estructura del SRS.	El documento conserva los RF registrados e identifica los apartados orientativos con la indicación de que están fuera del alcance de la tesis.
PB-14	Incorporar el contexto y la evidencia, y almacenar el documento.	El SRS incluye los fragmentos de respaldo y el contexto disponible, y se recupera mediante su asociación con el proyecto.
PB-15	Implementar la exportación a Word.	El archivo descargado puede abrirse y contiene los apartados y requisitos del documento generado.

Fuente: Elaboración propia.

Resultado del sprint: Se incorporó la generación de un documento SRS con estructura basada en IEEE 830-1998, que reúne los requisitos registrados, su evidencia y el contexto académico disponible. El documento se almacena y puede exportarse a Word; los apartados orientativos se identifican como fuera del alcance de la tesis.

3.7.5 Sprint 5: interfaz, integración y pruebas

Periodo: 1 al 7 de septiembre de 2026 (7 días calendario).

Objetivo: Completar el acceso a las funcionalidades desde la interfaz web e integrar el procesamiento de extremo a extremo.

Este sprint concentra la consulta de resultados, la conexión de los cuatro agentes con el orquestador y las comprobaciones del flujo completo. La revisión contempla tanto el procesamiento con entradas válidas como las situaciones que requieren comunicar un error al usuario.

Tabla 13.

Tareas y criterios de aceptación del sprint 5

ID	Tarea	Criterio de aceptación
PB-16	Completar la interfaz web.	El usuario gestiona proyectos y entrevistas y consulta transcripciones, RF, contexto académico y documentos SRS. El contexto aparece en la vista de requisitos.
PB-17	Integrar los cuatro agentes con la interfaz y el orquestador.	Una solicitud de procesamiento completo ejecuta las etapas en orden y permite consultar el SRS generado cuando el proceso finaliza correctamente.
PB-18	Comprobar el flujo completo y corregir problemas.	Se comprueban entradas válidas e inválidas, la interrupción ante errores no resueltos y la conservación de resultados. Los problemas detectados se documentan para su corrección y nueva comprobación.

Fuente: Elaboración propia.

Resultado del sprint: Se obtuvo un prototipo integrado con acceso desde la interfaz web. El usuario puede gestionar proyectos y entrevistas, ejecutar el procesamiento completo y consultar las transcripciones, los requisitos, el contexto académico y los documentos SRS.

El cierre de esta organización del desarrollo da paso a la evaluación del prototipo. Las evidencias de las pruebas, las métricas y el análisis de la calidad de los resultados se presentan en el capítulo IV.

3.8 Estructura del proyecto

El código del prototipo se organiza en directorios que agrupan los componentes según su responsabilidad. Esta distribución permite localizar la implementación de los agentes, la comunicación MCP, los servicios de apoyo y los elementos de la interfaz web.

3.8.1 Organización de directorios y archivos

La Figura 12 presenta una vista simplificada de los principales directorios y archivos del proyecto. Se omiten las dependencias del entorno virtual, los archivos temporales y otros elementos auxiliares para facilitar su lectura.



Figura 12.

Estructura de directorios del prototipo.

El directorio `app/` concentra el código de la aplicación. En su interior se encuentran los

módulos de procesamiento, las rutas, los modelos de datos y los recursos de presentación. Los archivos de configuración e inicio se ubican en la raíz del proyecto, mientras que la base de datos y los audios se conservan en directorios separados.

La estructura del proyecto se presenta en dos figuras para facilitar su lectura. La primera muestra los componentes de procesamiento y comunicación; la segunda reúne los servicios de apoyo, la interfaz web y los archivos de configuración.



Figura 13.
Estructura de servicios

3.8.2 Separación de responsabilidades en el código

La Tabla 14 relaciona los principales directorios y archivos con su función dentro de la implementación. Las rutas indicadas parten de la carpeta raíz del proyecto.

Tabla 14.

Responsabilidades de los directorios y archivos principales

Directorio o archivo	Responsabilidad
app/agents/	Contiene la implementación de los cuatro agentes especializados.
app/orchestrator/	Coordina la ejecución secuencial del procesamiento completo.
app/mcp/	Agrupar el cliente MCP y los servidores de consulta académica, persistencia y generación documental.
app/rag/	Organiza la recuperación y selección de publicaciones y la preparación del contexto para el modelo.
app/services/	Implementa el acceso a Whisper y Ollama, la consulta académica y los servicios de elaboración documental.
app/templates/	Contiene las plantillas de las páginas de la aplicación web.
app/static/	Almacena los recursos estáticos, incluidos los estilos CSS propios.
app/routes.py	Define las rutas que reciben las solicitudes y conectan la interfaz con las operaciones del sistema.
app/models.py	Define los modelos de datos y sus relaciones mediante SQLAlchemy.
app/__init__.py	Crea y configura la aplicación Flask, inicializa la base de datos y registra las rutas.
instance/ y uploads/	Conservan la base de datos SQLite y los archivos de audio, respectivamente.
config.py	Centraliza las opciones de configuración, las rutas de almacenamiento y las restricciones de carga de archivos.
requirements.txt	Registra las dependencias de Python y sus versiones.
run.py	Crea la instancia de la aplicación y permite iniciar su ejecución local.

Fuente: Elaboración propia.

Esta organización separa la presentación, la coordinación y los servicios de procesamiento. Por ejemplo, los cambios en la distribución visual se concentran en las plantillas y los estilos, mientras que los ajustes en las instrucciones de extracción se localizan en el agente correspondiente. De esta manera, la estructura facilita identificar los archivos involucrados en cada modificación.

3.9 Dificultades técnicas y soluciones implementadas

Durante el desarrollo se realizaron ajustes en la extracción de requisitos, la recuperación de información académica y la elaboración del documento SRS. Esta sección describe los problemas abordados y las soluciones incorporadas al prototipo. Sus efectos se presentan en términos de funcionamiento.

3.9.1 Control de la extracción de requisitos

La extracción mediante un modelo de lenguaje requiere controles para evitar que las respuestas incorporen funcionalidades sin respaldo en la entrevista, repitan requisitos o incluyan enunciados ajenos al alcance funcional. Una respuesta correctamente estructurada no garantiza, por sí sola, que su contenido represente lo solicitado por el entrevistado.

Para abordar esta dificultad, se delimitaron las instrucciones del agente de análisis y se solicitó una respuesta en formato JSON. Además, se incorporaron comprobaciones sobre los candidatos obtenidos, considerando su relación con el texto de origen y su similitud con los requisitos previamente aceptados. Cada requisito almacenado conserva el fragmento de la entrevista que respalda su identificación.

También se añadieron indicadores de posible ambigüedad y verificabilidad para apoyar la revisión del contenido. Estos controles permiten filtrar candidatos y conservar su trazabilidad, aunque no sustituyen la valoración de la calidad de los requisitos.

3.9.2 Ajustes en el enriquecimiento académico

La búsqueda general en la web no delimitaba las fuentes al tipo de publicaciones requerido para el contexto académico del proyecto. Por otra parte, generar un contexto independiente para cada requisito producía una presentación fragmentada y podía repetir información común al sistema analizado.

La solución consistió en orientar la recuperación automática hacia publicaciones académicas mediante Crossref. Las consultas consideran el dominio del proyecto y sus funcionalidades, mientras que la síntesis utiliza los resúmenes disponibles de las publicaciones seleccionadas. Google Scholar se mantiene como enlace de apoyo para la consulta manual.

La integración de Scopus se retiró de la búsqueda al no disponer de la clave necesaria para consultar su API. De este modo, el procesamiento dejó de depender de esa configuración y de mostrar la advertencia asociada a su ausencia.

Asimismo, el enriquecimiento se reorganizó como un resultado único por proyecto. El contexto se almacena y se presenta junto con los requisitos en la interfaz, sin repetir una síntesis para cada uno. Cuando las fuentes recuperadas no aportan evidencia suficiente, el sistema registra esta situación y permite continuar con la generación del documento.

3.9.3 Delimitación del contenido del documento SRS

La estructura basada en IEEE 830-1998 contempla apartados adicionales a los requisitos funcionales, mientras que el alcance de la tesis se concentra en su generación. Completar el documento exigía distinguir entre la información extraída de las entrevistas y el contenido elaborado como orientación.

Para resolver esta diferencia, se conservaron los requisitos funcionales registrados y sus fragmentos de respaldo. Los demás apartados se completaron mediante contenido orientativo relacionado con el proyecto, combinando plantillas y generación de texto. Estas secciones incluyen la indicación (*Fuera del alcance de la tesis*).

Esta distinción permite presentar una estructura documental más completa e identificar qué contenido requiere definición y revisión posterior. Los apartados orientativos no se consideran requisitos confirmados ni sustituyen la información que debe obtenerse de los participantes del proyecto.

CAPÍTULO IV

RESULTADOS Y ANÁLISIS

CONCLUSIONES

RECOMENDACIONES

BIBLIOGRAFÍA

- [1] E. Wolf, A. Trendowicz y J. Siebert, “Quality assessment of software requirements using artificial intelligence methods: A systematic literature review,” *Information and Software Technology*, vol. 191, pág. 107979, mar. de 2026, ISSN: 09505849. DOI: 10.1016/j.infsof.2025.107979
- [2] T. Rahman e Y. Zhu, “Automated User Story Generation with Test Case Specification Using Large Language Model,” abr. de 2024. dirección: <http://arxiv.org/abs/2404.01558>
- [3] J. Doe, “Recommended practice for software requirements specifications (ieee),” *IEEE, New York*, 2011.
- [4] I. C. S. S. E. T. Committee, *IEEE Guide to Software Requirements Specifications: An American National Standard*. IEEE, 1984.
- [5] M. A. Umar y K. Lano, “Advances in automated support for requirements engineering: a systematic literature review,” *Requirements Engineering*, vol. 29, págs. 177-207, 2 jun. de 2024, ISSN: 1432010X. DOI: 10.1007/s00766-023-00411-0
- [6] “Persona-driven automated extraction of non-functional requirements using LLM agents,” en *Procedia Computer Science*, vol. 270, Elsevier B.V., 2025, págs. 485-494. DOI: 10.1016/j.procs.2025.09.167
- [7] A. Akhtar y S. Akhtar, “Requirements Elicitation in Transition: A Review of Conventional and Contemporary Approaches,” *ICCK Journal of Software Engineering*, vol. 1, pág. 32, 1 ago. de 2025. DOI: 10.62762/jse.2025.862549
- [8] Z. Hoy y M. Xu, *Agile Software Requirements Engineering Challenges-Solutions—A Conceptual Framework from Systematic Literature Review*, jun. de 2023. DOI: 10.3390/info14060322
- [9] T. Guo et al., “Large Language Model based Multi-Agents: A Survey of Progress and Challenges,” *inf. téc.*, 2024.

- [10] P. P. Ray, *A Survey on Model Context Protocol: Architecture, State-of-the-art, Challenges and Future Directions*, abr. de 2025. DOI: 10.36227/techrxiv.174495492.22752319/v1 dirección: <https://www.techrxiv.org/users/913189/articles/1286748-a-survey-on-model-context-protocol-architecture-state-of-the-art-challenges-and-future-directions?commit=9234df64ba662b5f79543305ebc7334fbf834043>
- [11] P. Landeta-López, J. M. García, C. Guevara-Vega y A. Ruiz-Cortés, “Applying LLMOps to Support the Automatic Analysis of CAs,” 2024. DOI: 10.13039/501100011033/ERDF/EU dirección: https://doi.org/10.1007/978-981-96-7423-7_13
- [12] C. P. Guevara-Vega, E. D. Guzmán-Chamorro, V. A. Guevara-Vega, A. V. B. Andrade y J. A. Quiña-Mera, “Functional Requirement Management Automation and the Impact on Software Projects: Case Study in Ecuador,” en *Advances in Intelligent Systems and Computing*, vol. 918, Springer Verlag, 2019, págs. 317-324. DOI: 10.1007/978-3-030-11890-7_31
- [13] A. D. Toro y B. B. Jiménez, “Metodología para la elicitación de requisitos de sistemas software,” *Informe Técnico LSI-2000-10. Facultad de Informática y Estadística Universidad de Sevilla*, 2000.
- [14] T. Puchleitner, S. Lubos, A. Felfernig y D. Garber, “Evaluating Large Language Models for the Automated Generation of Software Requirements,” en *Lecture Notes in Computer Science*, vol. 15707 LNAI, Springer Science y Business Media Deutschland GmbH, 2026, págs. 443-450, ISBN: 9789819688913. DOI: 10.1007/978-981-96-8892-0_37
- [15] M. A. Sami et al., “A Multi-agent LLM System for Automated Requirements Analysis: A Study on User Story Generation and Prioritization,” en *Lecture Notes in Computer Science*, vol. 16082 LNCS, Springer Science y Business Media Deutschland GmbH, 2026, págs. 178-187, ISBN: 9783032041999. DOI: 10.1007/978-3-032-04200-2_12
- [16] V. Terragni, A. Vella, P. Roop y K. Blincoe, “The Future of AI-Driven Software Engineering,” *ACM Transactions on Software Engineering and Methodology*, vol. 34, 5 mayo de 2025, ISSN: 15577392. DOI: 10.1145/3715003

- [17] A. Nguyen-Duc et al., “Generative Artificial Intelligence for Software Engineering—A Research Agenda,” *Software - Practice and Experience*, vol. 55, págs. 1806-1843, nov. de 2025, ISSN: 1097024X. DOI: 10.1002/spe.70005
- [18] T. Mahbub, D. Dghaym, A. Shankarnarayanan, T. Syed, S. Shapsough e I. Zualkernan, “Can GPT-4 Aid in Detecting Ambiguities, Inconsistencies, and Incompleteness in Requirements Analysis? A Comprehensive Case Study,” *IEEE Access*, vol. 12, págs. 171 972-171 992, 2024, ISSN: 21693536. DOI: 10.1109/ACCESS.2024.3464242
- [19] M. Binkhonain y R. Alfayez, “Are prompts all you need? Evaluating prompt-based Large Language Models (LLM)s for software requirements classification,” *Requirements Engineering*, vol. 30, págs. 423-443, 4 dic. de 2025, ISSN: 1432010X. DOI: 10.1007/s00766-025-00451-8
- [20] R. M. Peña, A. Simón-Cuevas, F. P. Romero y J. A. Olivas, “Generation of Software Requirements from User Feedback Combining ML and LLMs,” en *Lecture Notes in Computer Science*, vol. 16328 LNCS, Springer Science y Business Media Deutschland GmbH, 2026, págs. 67-79. DOI: 10.1007/978-3-032-11358-0_6
- [21] S. Aboukadri, A. Ouaddah, A. Mezrioui e I. E. Asri, “Leveraging RAG and LLMs for Access Control Policy Extraction From User Stories in Agile Software Development,” *IEEE Access*, vol. 13, págs. 116 462-116 472, 2025, ISSN: 21693536. DOI: 10.1109/ACCESS.2025.3586203
- [22] Z. Zhang, M. Rayhan, T. Herda, M. Goisau y P. Abrahamsson, “LLM-based agents for automating the enhancement of user story quality: An early report,” en *International Conference on Agile Software Development*, ép. Lecture Notes in Business Information Processing, Springer, 2024, págs. 117-149. DOI: 10.1007/978-3-031-61154-4_8
- [23] X. Hou, Y. Zhao, S. Wang y H. Wang, “Model context protocol (mcp): Landscape, security threats, and future research directions,” *ACM Transactions on Software Engineering and Methodology*, 2025.
- [24] H. Cheng et al., “Generative ai for requirements engineering: A systematic literature review,” *Software: Practice and Experience*, vol. 56, n.º 2, págs. 141-170, 2026.

- [25] E. D. Canedo, A. T. S. Calazans, I. N. Bandeira, P. H. T. Costa y E. T. S. Masson, "Guidelines adopted by agile teams in privacy requirements elicitation after the Brazilian general data protection law (LGPD) implementation," *Requirements Engineering*, vol. 27, n.º 4, págs. 545-567, 2022.
- [26] M. A. Zadenoori, J. Dąbrowski, W. Alhoshan, L. Zhao y A. Ferrari, "Large language models (llms) for requirements engineering (re): A systematic literature review," *arXiv preprint arXiv:2509.11446*, 2025.
- [27] A. Hemmat, M. Sharbaf, S. Kolahtouz-Rahimi, K. Lano y S. Y. Tehrani, "Research directions for using LLM in software requirement engineering: A systematic review," *Frontiers in Computer Science*, vol. 7, pág. 1 519 437, 2025.
- [28] M. A. K. Raiaan et al., "A review on large language models: Architectures, applications, taxonomies, open issues and challenges," *IEEE Access*, vol. 12, págs. 26 839-26 874, 2024. DOI: 10.1109/ACCESS.2024.3365742
- [29] J. A. Khan, S. Qayyum y H. S. Dar, "Large language model for requirements engineering: A systematic literature review," 2025.
- [30] C. Ellsel y R. Stark, "Advancing Requirements Engineering with Large Language Models," *Procedia CIRP*, vol. 136, págs. 701-706, 2025.
- [31] Z. Ji et al., "Survey of hallucination in natural language generation," *ACM computing surveys*, vol. 55, n.º 12, págs. 1-38, 2023.
- [32] V. Rawte, A. Sheth y A. Das, "A survey of hallucination in large foundation models," *arXiv preprint arXiv:2309.05922*, 2023.
- [33] Y. Gao et al., "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, 2023.
- [34] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive nlp tasks," *Advances in neural information processing systems*, vol. 33, págs. 9459-9474, 2020.
- [35] Y. Huang y J. X. Huang, "A survey on retrieval-augmented text generation for large language models," *ACM Computing Surveys*, vol. 58, n.º 12, págs. 1-38, 2026.

- [36] Z. Zhang et al., “A survey on the memory mechanism of large language model-based agents,” *ACM Transactions on Information Systems*, vol. 43, n.º 6, págs. 1-47, 2025.
- [37] Z. Durante et al., “Agent ai: Surveying the horizons of multimodal interaction,” *arXiv preprint arXiv:2401.03568*, 2024.
- [38] Z. Xi et al., “The rise and potential of large language model based agents: A survey. arXiv 2023,” *arXiv preprint arXiv:2309.07864*, vol. 10, 2025.
- [39] Q. Wu et al., “Autogen: Enabling next-gen llm applications via multi-agent conversation,” *arXiv preprint arXiv:2308.08155*, 2023.
- [40] Anthropic, *Introducing the Model Context Protocol*, nov. de 2024. dirección: <https://www.anthropic.com/news/model-context-protocol>
- [41] E. Lumer, A. Gulati, V. K. Subbiah, P. H. Basavaraju y J. A. Burke, “Scalemcp: Dynamic and auto-synchronizing model context protocol tools for llm agents,” en *International Joint Conference on Computational Intelligence*, Springer, 2025, págs. 23-42.
- [42] G. Li, S. Liang, S. Nie, W. Liu, Z. Yang y L. Xiao, “Deep neural network-based generalized sidelobe canceller for robust multi-channel speech recognition,” en *Proceedings of the Annual Conference of the International Speech Communication Association, INTERSPEECH*, vol. 2020-October, International Speech Communication Association, 2020, págs. 51-55, ISBN: 9781713820697. DOI: 10.21437/Interspeech.2020-1101
- [43] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey e I. Sutskever, “Robust Speech Recognition via Large-Scale Weak Supervision,” dic. de 2022. dirección: <http://arxiv.org/abs/2212.04356>
- [44] A. Baevski, Y. Zhou, A. Mohamed y M. Auli, “wav2vec 2.0: A framework for self-supervised learning of speech representations,” *Advances in neural information processing systems*, vol. 33, págs. 12 449-12 460, 2020.
- [45] M. Richards y N. Ford, *Fundamentals of software architecture: an engineering approach*. O’Reilly Media, 2020.
- [46] S. Newman, *Building microservices: designing fine-grained systems*. O’Reilly Media, Inc., 2021.

ANEXOS